Acceleration for Deep Reinforcement Learning using Parallel and Distributed Computing: A Survey

ZHIHONG LIU, National University of Defense Technology, Changsha, China
XIN XU, National University of Defense Technology, Changsha, China
PENG QIAO, National University of Defense Technology, Changsha, China
DONGSHENG LI, National University of Defense Technology, Changsha, China

Deep reinforcement learning has led to dramatic breakthroughs in the field of artificial intelligence for the past few years. As the amount of rollout experience data and the size of neural networks for deep reinforcement learning have grown continuously, handling the training process and reducing the time consumption using parallel and distributed computing is becoming an urgent and essential desire. In this article, we perform a broad and thorough investigation on training acceleration methodologies for deep reinforcement learning based on parallel and distributed computing, providing a comprehensive survey in this field with state-of-the-art methods and pointers to core references. In particular, a taxonomy of literature is provided, along with a discussion of emerging topics and open issues. This incorporates learning system architectures, simulation parallelism, computing parallelism, distributed synchronization mechanisms, and deep evolutionary reinforcement learning. Furthermore, we compare 16 current open-source libraries and platforms with criteria of facilitating rapid development. Finally, we extrapolate future directions that deserve further research.

CCS Concepts: • **Computing methodologies** → **Concurrent algorithms**; **Reinforcement learning**; **Massively parallel algorithms**; *Distributed artificial intelligence;*

Additional Key Words and Phrases: Deep reinforcement learning, acceleration, parallel and distributed computing, large-scale

ACM Reference Format:
Zhihong Liu, Xin Xu, Peng Qiao, and DongSheng Li. 2024. Acceleration for Deep Reinforcement Learning using Parallel and Distributed Computing: A Survey. *ACM Comput. Surv.* 57, 4, Article 91 (December 2024), 35 pages. https://doi.org/10.1145/3703453

1 Introduction

Since the advent of **Deep Q Network** (**DQN**) [1] in 2015 which achieved human-level control on Atari video games, **deep reinforcement learning** (**DRL**), a powerful machine learning paradigm,

This work was supported by the National Natural Science Foundation of China under grant U21A20518, 62025208 and 62421002.
Authors' Contact Information: Zhihong Liu, National University of Defense Technology, Changsha, China; e-mail: zhliu@nudt.edu.cn; Xin Xu, National University of Defense Technology, Changsha, China; e-mail: xinxu@nudt.edu.cn; Peng Qiao, National University of Defense Technology, Changsha, China; e-mail: pengqiao@nudt.edu.cn; DongSheng Li, National University of Defense Technology, Changsha, China; e-mail: dsli@nudt.edu.cn.

ACM 0360-0300/2024/12-ART91
https://doi.org/10.1145/3703453

has been widely investigated by **Artificial Intelligence** (**AI**) researchers. DRL deals with training an agent to learn the optimal policy based on feedback from interaction with the environment. Through leveraging the power of **deep learning** (**DL**) and **reinforcement learning** (**RL**), there are many merits in DRL. First, DRL is able to handle high-dimensional and large state spaces. Second, DRL has the general self-learning ability without the requirements of labeled datasets. Third, problems that need to be solved by online computation traditionally can be solved in offline training manner with DRL. With these merits, DRL in recent years has led to dramatic breakthroughs in game playing [2–4], robotics [5–8], healthcare [9–11], and so on.

However, with the increased complexity of the applications for DRL, more interaction iterations and larger scale of neural network models are required. As a result, the training process becomes very computation-intensive and thus time-consuming. For instance, it needs around 38 days of experience to train DQN on Atari 2600 game [1] with 50 million frames. Besides, tuning of hyperparameters is tricky and further increases the time consumption [12].

Thus, it is intuitive to leverage parallel and distributed computing in DRL to accelerate its training process. During the past few years, rapid progress has been witnessed in this field, resulting in a profusion of studies that target diversified aspects and use a variety of techniques. This causes significant difficulties in understanding the development of this field in a systematic manner.

In this survey, we collect, classify, and compare a huge body of work on DRL acceleration using parallel and distributed computing, providing a comprehensive survey in this field with state-of-the-art methods and pointers to core references. In particular, we analyze the primary challenges to make DRL training distributed and demystify the details of the technologies that have been proposed by researchers to address these challenges. This includes system architectures for distributed DRL, simulation parallelism, computing parallelism, distributed synchronization mechanisms (*backpropagation-based training*), and deep evolutionary RL (*evolution-based training*). Furthermore, we provide an overview and comparison of current open-source libraries and platforms that put parallel and distributed DRL into practice. Based on that, we extrapolate potential directions for future work.

1.1 Related Surveys

There are a number of surveys in this field that are related to ours. Arulkumaran et al. [13] provide a brief survey on DRL algorithms, applications, and challenges. Li [14] gives a wide coverage of core elements, important mechanisms, and applications in DRL. Wang et al. [15] present an overview of the theories, algorithms, and research topics of DRL. Moerland et al. [16] provide the taxonomy, key challenges, and benefits of model-based RL. Meanwhile, many surveys targeting specific application domains have emerged in recent years, e.g., autonomous driving [17], cyber security [18], networking [19], intelligence transportation [20], multi-agent systems [21], and Internet of Things [22]. However, these surveys either focus on the fundamental theories and general technologies of DRL or investigate DRL methods in specific domains. Few of them studies on learning acceleration by parallel and distributed computing.

Admittedly, there are several surveys have been proposed in distributed machine learning [23–25]. It is noted that RL is one of the sub-branches in machine learning. These surveys, however, summarize distributed technologies from a holistic perspective of machine learning, without in-depth discussion regarding distributed RL. Besides, many works have emerged on distributed DL training acceleration. Mayer et al. [26] present a comprehensive investigation on challenges, techniques, and frameworks for DL training on distributed infrastructures. Ben-Nun et al. [27] provide a concurrency analysis for parallel and distributed DL. Tang et al. provide a comprehensive survey of distributed DL from the communication perspective. Chahal et al. [28] propose a hitchhiker's guide on distributed training for DL. However, these surveys target DL and have not analyzed any

methods in DRL. Note that DL and DRL are different types of machine learning paradigms, and the key difference exactly lies in the training scheme. With respect to DL, the methods learn from supervised training datasets and seek to minimize the error between the neural network model and the labeled data. In comparison, DRL methods require interaction with environments to generate experiences. Based on that, DRL methods learn what actions lead to the maximum outcome. This will unavoidably cause differences in training acceleration techniques between DL and DRL.

In particular, Samsami et al. [29] provide a brief overview of distributed DRL by introducing several key research works in this field. No taxonomy of existing methods is presented in their work. Besides, Yin et al. [30] propose a survey on distributed DRL for the specific field of multi-player multi-agent learning, where agents are either cooperating or competing within the environments. In contrast, our survey takes a much more comprehensive and in-depth view on training acceleration by parallel and distributed computing. Along with the outlined taxonomy and literature overview, the details of key methodologies in DRL training are demystified. To the best of our knowledge, this is the first comprehensive survey on acceleration for DRL using parallel and distributed computing.

1.2 Structure of the Survey

The structure of the survey is summarized in Figure 1 and organized as follows:

— Section 2 provides the foundations and challenges related to our study.
— Section 3 classifies system architectures in general for parallel and distributed DRL.
— Section 4 elaborates on different simulation parallelism strategies to expose simulation concurrency.
— Section 5 analyzes computing parallelism patterns used in existing works.
— Section 6 discusses distributed synchronization mechanisms for backpropagation-based distributed training methods.
— Section 7 explores technologies of deep evolutionary RL for evolution-based distributed training methods.
— Section 8 compares current open-source libraries and platforms that put parallel and distributed DRL into practice.
— Section 9 gives concluding remarks and highlights future directions that deserve further research.

2 Background

DRL integrates DL and RL, where DL is leveraged to perform function approximation in RL. Thus, the basic knowledge of DRL and current distributed DL training development lay a foundation for parallel and distributed DRL. This section will discuss the foundations related to this work, and analyze its challenges to motivate this study.

2.1 Preliminaries of DRL

Formally, DRL can be modeled as a **Markov Decision Process** (**MDP**). More specifically, at each time step t, the DRL agent receives a state $s_t \in \mathcal{S}$ and makes its action decision $a_t \in \mathcal{A}$ according to a distribution called policy $\pi(\theta|s_t)$, where $\mathcal{S}$ and $\mathcal{A}$ are the state space and action space, respectively. The policy is basically modeled by a neural network with parameters θ. After the action execution, the agent receives a reward r_{t+1} and transitions to the next state s_{t+1} according to the reward function $R(s_t, a_t, s_{t+1})$ and transition distribution $P(s_{t+1}|s_t, a_t)$, respectively. The return starting from time t to the end of the interaction is defined by $G_t = \sum_{i=0}^{\infty} \gamma^i r_{t+i}$, where $\gamma \in [0, 1]$ is the discount factor. The semantics of DRL are illustrated in Figure 2. DRL agents interact with

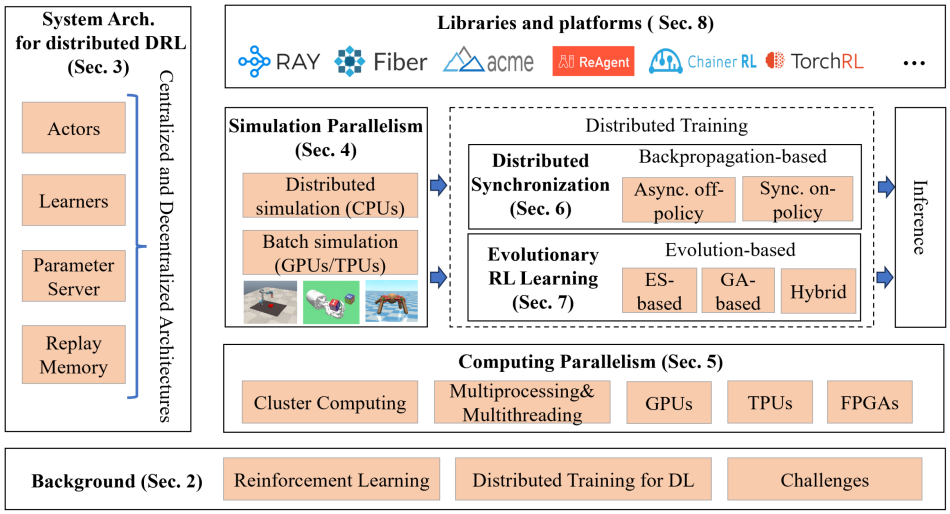


Fig. 1. The structure of the survey.

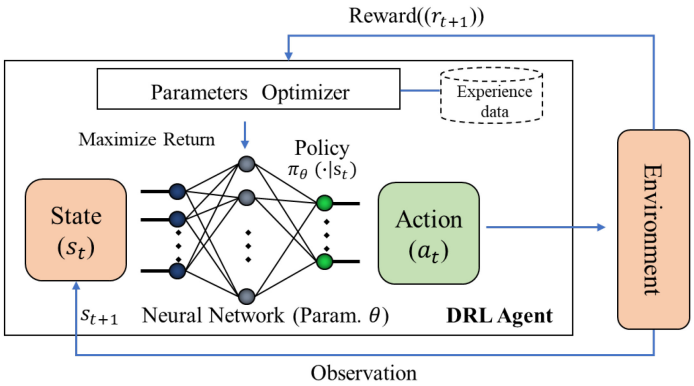


Fig. 2. The semantics of DRL.

the environment to collect experience data, which is usually denoted by $(s_t, a_t, s_{t+1}, r_{t+1})$. Based on experience data, the agent is trained to learn an optimal policy for maximizing the return by updating the parameters θ of the neural network.

DRL can be classified as *model-based* and *model-free*. In *model-based* variants, the transition and reward models (also named MDP dynamics) are known or learned first. Then the policy optimization is based on the models. There are mainly two categories in *model-based* DRL [16]. First is *model-based DRL with a learned model*. This category of methods learns both the model and the global policy. Examples are Dyna-style algorithms such as MB-MPO [31] and ME-TRPO [32]. These algorithms learn the model through data from interaction with environments. Then model-free algorithms are utilized to learn the global policy based on the data generated by the learned model. Second is *model-based DRL with a known model*. This category of methods plans over a known model, and only performs learning for the global policy. Examples are dynamic programming [33] and Monte-Carlo tree search-based algorithms (AlphaZero [3]).

In contrast, *model-free* DRL follows trail-and-error patterns and learns the policy directly. Basically, there are mainly three categories in *model-free* DRL. First is *value-based methods*. This

category learns a value function that represents the expected return for being in a state or taking an action in the state, e.g., $V_\pi(s)$ and $Q_\pi(s, a)$. Then an optimal policy can be obtained based on the value function. Value-based methods are suited for discrete action spaces and deterministic policies. Second is *Policy-based methods*. This category learns the policy directly and extracts actions according to the policy. Since the policy is actually a distribution of possible actions, continuous action spaces, and stochastic policies can also be learned. Compared to value-based methods, policy-based methods have better convergence properties. However, high variance and sample inefficient are common issues in policy-based methods. Third is *Actor-critic-based methods*, which integrate the merits of value-based and policy-based methods. Two separate neural networks are used in the architecture, named actor and critic. The actor learns the optimal policy and chooses the action by using the policy-based methods. The critic evaluates the selected action through the value-based methods. Due to the superiority in performance, actor-critic-based methods have become more and more popular in recent years, e.g., PPO [34], SAC [35], and TD3 [36].

Alongside supervised learning and unsupervised learning, RL performs optimization based on the experiences collected from interacting with uncertain environments. Finding a balance between *exploration* (to discover new features about the world) and *exploitation* (to utilize what it knows) is fundamental to the efficiency of RL methods. Meanwhile, according to the uniformity of the behavior policy and the target policy, DRL algorithms can be divided into *on-policy* and *off-policy*. Here, the behavior policy represents the policy used to generate the training data by interaction, while the target policy refers to the policy that needs to be learned. More specifically, in *off-policy* learning, the behavior policy and the target policy are different. The goal of the off-policy learning is to learn the target policy by using the training data generated by the behavior policy. This has many benefits such as learning from demonstration, continuous exploration, and better sample utilization. However, it suffers from convergence and stability issues [37, 38]. In comparison, the behavior policy and the target policy are the same in *on-policy* learning. As a result, better stability can be achieved in learning but sacrificing sample efficiency [35].

2.2 Distributed Training for DL

In recent years, distributed training acceleration for DL has become a hot research topic and various methods have flourished in this field. Due to the tight connection between DL and DRL, knowledge and progress of distributed DL methods play a foundational role in parallel and distributed DRL. Basically, there are mainly three parallelism schemes of training acceleration for DL: data parallelism, model parallelism, and pipeline parallelism.

Data parallelism [39–43] is the first proposed and the most common parallelism scheme. It partitions the training dataset (samples) into multiple subsets and dispatches them to multiple workers (e.g., compute nodes and devices). Each worker maintains a replica of the entire neural network model along with its local parameters. During training, each worker processes its subset of data independently and synchronizes parameters or gradients of the model across different workers either in a synchronous or asynchronous manner. This is the main parallelism scheme utilized in distributed DRL. Details can be found in Section 6.

Model parallelism [44–47] slices the neural network model into disjoint partitions and assigns them to multiple workers for computation. Compared to data parallelism, the computations between workers are no longer independent. Neurons with connections across workers require data transfers before continuing the computation to the next layer. Due to the high computation dependency in neural networks, model parallelism is non-trivial for accelerating the training [48]. The main advantage of model parallelism is making training a large model, which cannot fit into the memory of one node, become possible.

Pipeline parallelism [49–52] divides the neural network model into stages (consisting of consecutive layers) and assigns the stages to different workers. Each worker performs the computation of its stage for microbatches of samples one by one in a pipeline fashion. This scheme can significantly improve parallel training throughput over the vanilla model parallelism by fully saturating the pipeline. Basically, pipeline parallelism is a special form of model parallelism.

Combining these parallelism schemes therefore giving full play to their respective advantages is a promising direction [53–59]. Krizhevsky et al. [53] propose a hybrid scheme of using data parallelism for convolutional layers and model parallelism for densely-connected layer. Wu et al. [54] propose to make use of data and model parallelism to accelerate the training of recurrent neural networks. Rasley et al. propose ZeRo [57], a parallelized optimizer based on model and data parallelism, which can train large models with over 100 billion parameters. Park et al. propose HeiPipe [58] that combines pipeline parallelism with data parallelism for training large deep networks.

2.3 Challenges of Parallel and Distributed Training for DRL

Although the above-distributed training methods can serve as good references for training acceleration in DRL, copying these methods to DRL directly is infeasible. That is because there are differences in training schemes between DL and DRL. DL trains from supervised training datasets and seeks to minimize the error between the neural network model and the labeled data. In comparison, DRL does not have labeled data. It learns an optimal policy that leads to the maximum outcome through interacting with the environment. This will unavoidably cause differences in training acceleration between DL and DRL and trigger requirements of new techniques and methodologies. Overall, the challenges of parallel and distributed training for DRL can be summarized as follows:

— Training DRL agents in parallel is a complex and dynamic problem that many components, such as actors, learners, and parameter servers, need to operate cooperatively. How to organize the architecture of these components and allocate the training tasks among them so as to maximize the system throughput is a challenging problem [60, 61].
— Requiring to interact with environments (mostly simulated) to generate training samples is a distinguished feather of DRL. Training DRL agents for complex applications often need millions or even billions of experience samples. How to parallel simulations so as to increase the sample efficiency is demanding [62, 63].
— The workloads in distributed DRL training are heterogeneous, which may cause data movement across devices even nodes frequently. This will degrade the overall throughput and computation efficiency. How to saturate the computation resources by using different hardware architectures and corresponding computing parallelism techniques accordingly is challenging [64].
— Since the parallel learning machines may process at different speeds in a heterogeneous environment, obsolete gradients exist and can have a large impact on stability and convergence in training. How to aggregate the gradients and synchronize the model maintained in parallel workers is also non-trivial [65].
— The essence of dominant training technology in distributed DRL is stochastic gradient descent based on backpropagation. However, this technology still suffers from local optima and expensive computational cost [66]. How to exploit other optimization or search techniques in DRL to enable more sufficient training is desired.

In addressing these challenges, a profusion of studies has been proposed in the field of DRL acceleration using parallel and distributed. There is a necessity to collect, classify, and compare these studies in a structured manner, thereby facilitating researchers understanding the development of this field. In the following sections, we survey the research for parallel and distributed DRL

training and demystify the details of the key technologies. This includes system architectures for distributed DRL, simulation parallelism, computing parallelism, distributed synchronization mechanisms (*backpropagation-based training*), and deep evolutionary RL (*evolution-based training*). In addition, we provide an overview and comparison of current open-source libraries and platforms that put parallel and distributed DRL into practice.

3 System Architectures for Distributed DRL

Training DRL agents in parallel is a system engineering problem that many components need to operate cooperatively. Taking a panoramic view of the parallel and distributed DRL acceleration frameworks that bloomed in recent years, four components can be abstracted in the general case:

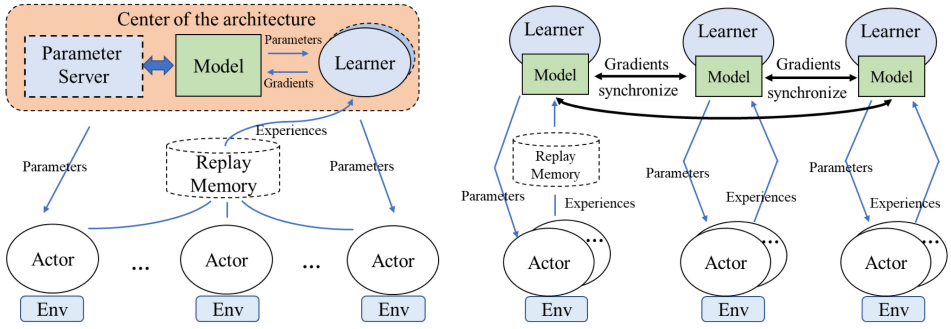
- *Actor*: is in charge of interacting with the environment, including performing the actions obtained according to the policy, getting information such as observations and rewards, and producing experience data.
- *Learner*: collects or samples the experience data, and computes the gradients for updating the model. Since the gradient computation is usually based on a batch of experiences, high-speed hardware like GPU is commonly used in Learners.
- *Parameter Server*: Maintains the up-to-date parameters of neural network models for Actors or Learners. Note that in some approaches (e.g., APE-X [67] and R2D2 [68]), the job of the parameter server is concurrently taken by the Learner.
- *Replay memory*: Stores the experience data produced by Actors. This exists in cases of off-policy RL, e.g., Gorila [69], Ape-X [67], R2D2 [68], and so on.

According to the organization of these components, we classify the parallel and distributed training architectures for DRL into two classes: centralized and decentralized architectures.

3.1 Centralized Architecture

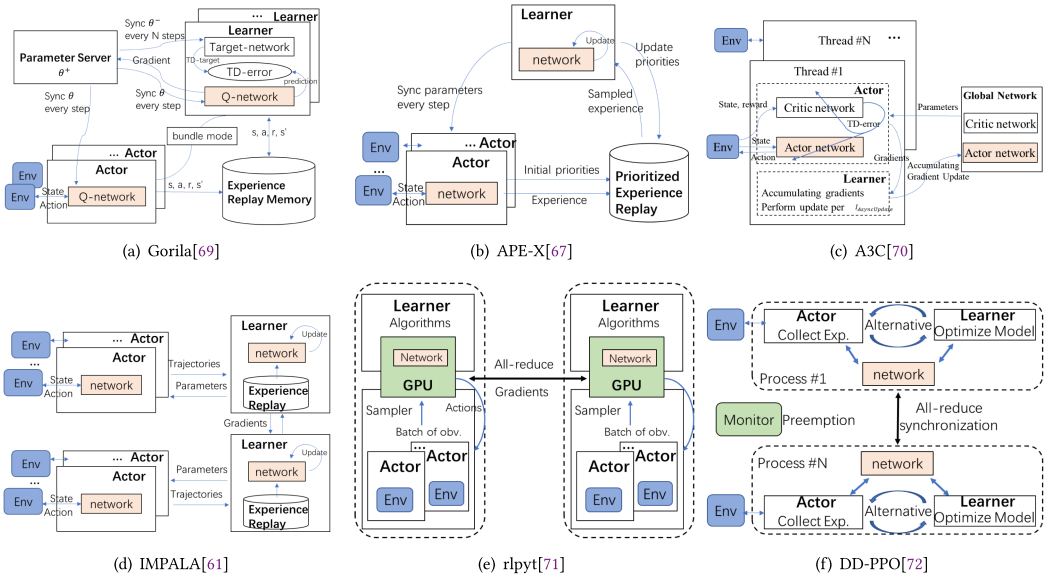
In the centralized architecture, there is a center that maintains the global model of the neural network for learning. Learners optimize the global model by computing gradients based on the experiences from Actors. Both Learners and Actors synchronize their local model from the global model in some period or steps. Compared to Actors who usually have a large scale, the number of Learners can be one or many. We abstract the general form of the centralized architecture as shown in Figure 3(a). The global model is either maintained by the Parameter Server or a Learner. More specifically, if there are multiple Learners in the architecture, the Parameter Sever serves as the center for updating the global model and distributing it to Actors and Learners. Therefore, the star communication topology is constructed in the centralized architecture. In any event, since a large number of workers as well as iterations could be involved in distributed DRL training, the bottleneck of the center node will have a strong impact on the training scalability.

Silver et al. [69] propose a massively distributed method for the DQN named Gorila (General RL Architecture). To the best of our knowledge, this is one of the pioneering works in the area of distributed training for DRL. Gorila is based on the centralized architecture and consists of four components: *Actors*, *Learners*, the *Parameter Server*, and the *Replay Memory*, as shown in Figure 4(a). The center of Gorila is the Parameter Server, which is in charge of updating and distributing the network parameters. There can be many Actors and Learners in Gorila. Each Actor has a replica of the Q network and is responsible for generating experiences to the Replay Memory through interacting with the environment. Each learner also has a replica of the Q network and is in charge of producing the gradients to the parameter server based on the experiences from the Replay Memory. With the help of parallel Actors and Learners, Gorila can significantly improve the training performance for DQN, reaching a reduction of ten times in training duration on most games of Atari.



(a) Centralized architecture (b) Decentralized architecture

Fig. 3. Comparison of centralized and decentralized architectures for parallel and distributed training in DRL.



(a) Gorila[69] (b) APE-X[67] (c) A3C[70]

(d) IMPALA[61] (e) rlpyt[71] (f) DD-PPO[72]

Fig. 4. Architectures of current parallel and distributed DRL methods.

Some variants over Gorila have also been proposed to further improve the performance. Horgan et al. [67] propose a distributed method in centralized architecture for DQN with prioritized experience replay named APE-X. Rather than sampling uniformly in Gorila, APE-X focuses on learning the prioritized experiences with larger absolute **temporal difference** (**TD**) errors, as shown in Figure 4(b). Furthermore, a distributed method named **Recurrent Replay Distributed DQN** (**R2D2**) [68] is proposed based on APE-X, which imports the recurrent neural network (LSTM) in training to achieve better performance in the partially observed environment. These two methods are based on a similar architecture to Gorila. However, only one Learner is used in these two methods, which is running on a GPU. By doing this, the Learner can exploit the batch computing advantages of GPU for gradient computation compared to Gorila. Besides, they leverage the single Learner to maintain the latest parameters instead of the Parameter Server.

Unlike Gorila which is for off-policy learning, Minh et al. [70] propose a new type of distributed training method named A3C for on-policy learning. The architecture of A3C is shown in Figure 4(c). We can see that A3C is based on a centralized architecture. Different from the aforementioned methods, A3C does not utilize the Replay Memory. Instead, A3C leverages the parallelization of Actors to enrich the diversity of environmental interaction experiences and accumulates updates over multiple steps to improve the stability of learning. Besides, the Actor and Learner are encapsulated together in an actor-learner thread. Furthermore, although there is no Parameter Server, a global network is also maintained on a worker, which is considered as the center of the architecture. Each actor-learner thread sends accumulated gradients to and obtains the latest network parameters from the global network.

3.2 Decentralized Architecture

In the decentralized architecture, there is no central node that maintains the global model. Basically, more than one Leaner is deployed. Each Learner not only computes the gradients based on the experiences of the corresponding Actors but also obtains the gradients from other learners. Then, each Learner updates its model by aggregating all the gradients. Here, All-reduce [73, 74] distributed communication mechanism is utilized to aggregate the gradients, which generally requires a fully connected communication topology for parallel workers. Other than interacting with environments and producing the experiences, Actors update the parameters from Learners. We abstract the general form of the decentralized architecture as shown in Figure 3(b). Compared to the centralized architecture, by using multiple Learners to aggregate the gradients and maintain the model, the decentralized architecture removes the communication congestion on the Parameter Server [74].

A decentralized architecture of multiple learners with importance weighted is proposed in IMPALA [61]. In IMPALA, actors utilize CPU cores to interact with the environment, while learners are deployed on GPUs to give play to the gradient computation. To maintain the latest model in the whole system, gradients are aggregated across multiple learners synchronously and actors obtain the latest parameters from learners. Stooke et al. [71, 75] propose an acceleration method of utilizing multiple GPUs for DRL named *rlpyt*, which is based on the decentralized architecture. There are multiple Learners in the architecture. Each of them is running on a GPU and is in charge of data sampling and model inference. Besides, All-reduce is performed in each iteration to aggregate the gradients so as to keep the model in every Learner identical. This method has many variants that are applied with different DRL algorithms including A3C, PPO, DQN, and so on. A similar architecture is adopted in DD-PPO [72], which is also decentralized and leverages multiple GPUs. Other than the method in [75] that synchronizes the models of different GPUs in one machine, DD-PPO extends the architecture across many machines and has better scalability. Similar to A3C [70], DD-PPO encapsulates the work of the Actor and Learner together in a worker. In this way, each worker alternates the workloads of experience collection, gradient aggregation, and optimization to achieve better resource utilization.

3.3 Discussion

These two classes of architecture are commonly used in recent studies. In the centralized architecture, it is known that the central node may encounter communication congestion that limits the scalability of distributed training [23]. However, its merits on synchronization efficiency and realization simplicity are non-negligible. This is because the global updated model is maintained in a central node, from which all workers can easily obtain and keep consistent. In the decentralized architecture, the scalability issue is relieved by eliminating the single-point issue. Nevertheless, the models optimize separately and need to be synchronized through assembling, which may incur convergence latency and communication overhead [74].

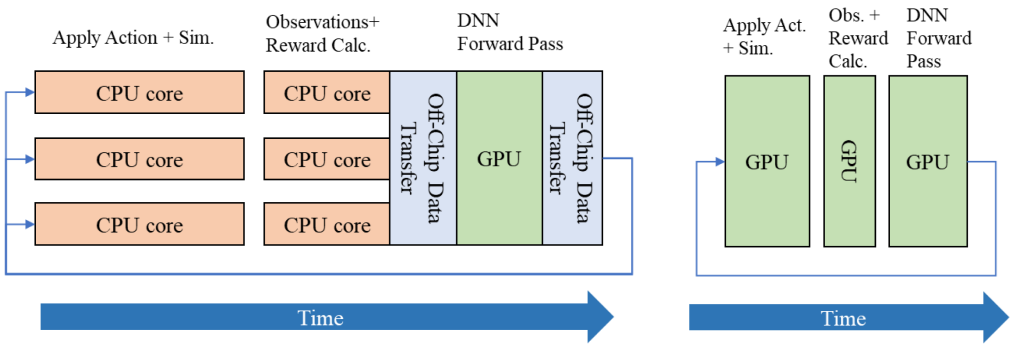
Overall, there are many open issues and emerging topics from the perspective of the learning system architecture. Firstly, the question of whether Actors should conduct model inference to derive actions remains unresolved. In scenarios where Actors are tasked with model inference, Actors need to maintain the up-to-date copy of the model, obtain the actions through model inference, and transfer the experiences after environment interaction (e.g., IMPALA shown in Figure 4(d)). Conversely, in other approaches, Actors focus on the interaction with the environment and give up the work of model inference (e.g., rlpyt shown in Figure 4(e)). Instead, Learners obtain the actions through model inference and transfer them to Actors. Both paths of solutions have two sides. The former path takes full advantage of computational capacities in Actors to share the responsibilities of model inference. However, frequent model update requests from many Actors will significantly decrease the learning efficiency of Learner [76]. The latter path reduces the overhead of model synchronization among Actors and Learners, but it may incur latency on transferring actions and experiences [77].

Besides, unlike the typical design of the system architecture where Actor and Leaner are the two types of distributed workers, some methods innovatively extend the workers to fine-grained types. For example, there are *Actor worker*, *Policy worker*, and *Trainer* in [78]. The *Actor worker* is responsible for environment interaction, the *Policy worker* takes charge of policy inference, and the *Trainer* is in charge of model learning. The main purpose of this design is to increase computational efficiency by allocating different workers to different computing nodes that suit their tasks, e.g., CPU, GPU, or TPU nodes. A similar design can be found in SampleFactory [60], where there are *Rollout worker*, *Policy worker*, and *Leaner* for the fine-grained types of parallel workers. Also, Li et al. [79] refined the workers to *Actor*, *V-learner*, and *P-learner* for data collection, value learning, and policy learning, respectively. Since there is a model learning job for Dyna-style *model-based* DRL, Zhang et al. [80] refine the parallel workers to *Data Collection Worker*, *Model Learning Worker*, and *Policy Improvement Worker* in their asynchronous *model-based* training method.

Furthermore, as previously discussed, the centralized architecture adheres to a star communication topology, whereas the decentralized architecture employs a fully connected communication topology. Each of these solutions carries its own drawbacks, such as the single point of failure in the centralized architecture and the increased synchronization overhead in the decentralized one. To achieve a better tradeoff, Assran et al. [81] propose a gossip-based Actor-Learner architecture where parallel workers are organized in a peer-to-peer communication topology. Gossip [81, 82] communication mechanism is used to exchange the model update in this method. Each parallel worker only communicates with its neighbors and does not need to communicate with everyone else. The theoretical proof is given that the parallel workers' models are guaranteed to remain within $\epsilon - close$ during training. Experiments also show that this architecture achieves competitive performance compared to A3C [70] and IMPALA [61]. Similarly, Sha et al. [83] propose distributed asynchronous policy evaluation based on directed peer-to-peer networks, allowing each parallel worker to update its value function locally by using data transferred from its neighbors.

4 Simulation Parallelism

Requiring to interact with the environment to collect training samples is one of the main differences between DRL and DL. Simulations that simulate realistic environments are widely used to reduce training cost and time, especially for physics-based applications, e.g., robotics and autonomous driving. There are many simulation platforms that are popular in DRL training, e.g., OpenAI Gym, MoJoCo, Surreal, Unity ML, Gazebo, and AirSim. Basically, the computations in simulations (e.g., physics calculation, rewards calculation, and 3D rendering) are involved in each step of training iteration. As the size of experiences increased for training complex applications, the



(a) CPU simulation parallelism pipeline

(b) GPU simulation parallelism pipeline

Fig. 5. Comparison of two simulation parallelism pipelines. (*a*) The intermediate data needs to be copied from CPUs to GPUs back and forth during training in the CPU simulation parallelism pipeline. (*b*) GPU simulation of zero-copy enables directly accessing to simulation results in the GPU buffers and keeps all of the computations on the GPU [62].

simulation efficiency becomes more and more important in the training acceleration. For instance, 2.5 billion frames of experience are needed for PointGoal navigation in 3D scanned environments [72]. How to parallel simulations so as to increase the sample efficiency is challenging. To this end, many approaches have been proposed. According to existing RL literature, there are mainly two simulation parallelism strategies.

4.1 Distributed Simulation Based on CPU Clusters

Many simulation platforms in use today are run on CPUs for many existing DRL methods in training, e.g., Atari in [1, 70, 84, 85], MoJoCo in [36, 86–88], and OpenAI Gym in [73, 89, 90]. A straightforward strategy for training acceleration is employing a cluster of CPUs to execute a number of instances of the environment in a distributed manner, where each instance normally runs on a process or a thread. Based on this, the experience data used for training can be generated at a higher speed. Besides, the experience data can also be more diversified by importing stochastic or using different policies when interacting with different instances of the environment. This strategy is adopted by many existing parallel and distributed DRL training methods such as Gorila [69], A3C [70], APE-X [67], and IMPALA [61], and yields impressive results. For example, in APE-X, approximately 50K environmental **frames per second** (**FPS**) can be generated by using 360 actor machines; each actor runs an instance of the environment by a CPU core.

Nevertheless, the scaling of these distributed simulation methods is determined by the number of CPU cores in the system. As the number of CPU cores and nodes in the cluster increases, potential overhead may be incurred in communication across nodes [91, 92], synchronization [72, 93], and resource allocation [94, 95]. Furthermore, as GPUs are commonly used in neural network computations, a combination of CPUs and GPUs is popular in most of the DRL researches. In this case, additional context-switching overhead would be introduced, as shown in Figure 5(a). That is because intermediate data needs to be copied from CPUs to GPUs back and forth during training in this case. It is noted that simulation is only one part of the training platform. After a simulation step in the environment, the next state and a reward need to be computed. When using GPUs for neural network computations, the experience data (e.g., the next state and rewards) needs to be transferred from system memory to GPU memory for neural network inference. Then, actions are produced and need to be copied again to system memory for CPUs to perform the simulation step.

4.2 Batch Simulation Based on GPUs/TPUs

In view of the limitations of CPU simulations on a large-scale, researchers have been committed to developing batch simulation methods based on specialized hardware architectures such as GPUs or TPUs. Liang et al. [96] propose a GPU-accelerated RL simulator to parallel the simulations, which can achieve thousands of humanoid robots concurrently simulated and generate 60 K environmental FPS in a single machine with one GPU and CPU. With the batch simulation framework, the humanoid robots can be trained to run in less than 20 minutes while using 1,000 × less CPU cores compared to previous works [97]. While relying on GPU to perform most of the simulation computations, this method still requires CPUs to perform tasks of getting state and applying controls. The performance of this method is also bounded by CPU-GPU communication bandwidth.

Therefore, zero-copy batch simulation that eliminates CPU-GPU communication has become a hot research topic in the field. Dalton et al. [94] propose a CUDA Learning Environment named CulE to support GPU emulation for Atari. CuLE enables the Atari simulation to run directly within GPU memory, thereby eliminating off-chip communications that previous methods endured [96]. Up to 155 K FPS in emulation only is achieved when emulated by CuLE with 4096 environments in parallel by 1 GPU. Furthermore, other than targeting 2D Atari simulation, Shacklett et al. [98] propose a large batch simulation method for 3D environments (PointGoal navigation). This method bundles a large batch of requests for simulations and processes one entire batch at once. The key idea is to amortize the costs of simulation computation, synchronization, and communication across a batch of requests. Similarly, Makoviychuk et al. [62] propose a high-performance robotics simulation platform named Isaac Gym for a variety of robotics environments. Isaac Gym offers a Tensor API that grants direct access to simulation results stored in GPU buffers, ensuring that all computations remain within the GPU. In this way, the bottleneck of off-chip communication between CPUs and GPUs can be eliminated, as shown in Figure 5(b). Isaac Gym enables thousands of environments simulated on a single GPU and achieves up to 300× improvement in the training time over previous work [99].

Besides, Freeman et al. [100] from Google Research propose an open-source library named Brax for large-scale rigid body simulation by using TPUs. Brax enables simulation computation and RL optimizer performed together on the same TPU. The experiment results show that Brax can achieve hundreds of millions of steps per second for MujoCo-ant on a TPUv3 8×8 accelerator.

4.3 Discussion

Currently, using a combination of CPUs and GPUs in DRL training is the mainstream in the field. CPUs are used to simulate the environments and perform environmental interaction, while GPUs are used to perform neural network inference as well as weights update. Hence, CPU-GPU communication is an unavoidable performance bottleneck. To this end, zero-copy simulation that contains all the computation in GPUs or TPUs is one of the significant research directions for highly efficient simulations. This provides an alternative way of simulation parallelism, with no requirement of accessing to CPU clusters which most researchers find hard to get. Besides, different from previous works that run each environment instance in the individual simulation, sharing scenes with other robots in a simulation is an effective way to take advantage of batch parallelism and maximize the throughput [62, 96, 98]. Since texture and geometry objects tend to be large in size, naively loading these objects for every environment is unaffordable for GPU memory. For instance, Liang et al. [96] loads all agents (and task-related objects) in the same simulation. Shacklett et al. [98] maintain $K \ll N$ unique scenes in GPU memory, where N is the number of parallel environments in a batch.

Transferring the learned policies from simulation to reality is an essential demand for many real-world applications. Simulation-to-reality (Sim-to-real) is a topic of much interest. One

straightforward strategy is to build a realistic simulator that can perfectly replicate the reality. Hence, the learned policies can be directly applied in real-world environments. This is referred to as the *zero-shot transfer*. In recent years, the *zero-shot transfer* has been successfully demonstrated in several domains. Andrychowicz et al. [6] transfer learned policies for dexterous in-hand manipulation to physical robots by performing training entirely in simulation. Rudin et al. [101] demonstrate sim-to-real transfer for quadrupedal robots by using massively parallel DRL. Loquercio et al. [102] achieved the *zero-shot transfer* for high-speed UAV navigation while some realistic scenarios were never experienced during training in simulation. The techniques for enabling seamless transfer in these approaches can be summarized as follows. First, add realistic sensor noise to the observations. Second, randomize physical properties in simulations such as friction coefficients and dynamics of objects. Third, learn with disturbances such as pushing the robot randomly and adding noise to rewards during training.

5 Computing Parallelism

Other than simulation, intensive computing workloads are unavoidable in DRL training e.g., network inference, backpropagation, and evolutionary computation (see Section 7). As a result, computing parallelism techniques are widely utilized to speed-up computations in DRL training. There are mainly three ways in the literature: cluster computing, single machine parallelism, and specialized hardware architectures (i.e., GPUs, FPGA, TPUs) acceleration.

5.1 Cluster Computing

Cluster computing usually includes multiple machines working together to perform computation-intensive or data-intensive tasks. Those machines are also called "nodes" and inter-connected by a high-speed local network. Since the computing resources in individual nodes are relatively constrained, integrating the resources of multiple nodes is a straightforward and effective way for acceleration of large-scale processing. In the early days when deep neural networks were not widely used in RL, the classic cluster computing framework (i.e., MapReduce [103]) was applied to RL for acceleration [104]. This method distributes the computation in the large matrix multiplication for MDP solutions such as policy evaluation and iteration. Considering the inadequacy of MapReduce for iterative computation involved in neural network training, Dean et al. propose DistBelief [44] to utilize clusters of machines in training and inference for large-scale deep networks. However, this method targets deep network training, which is different from that of DRL. To this end, one of the earliest works on parallel and distributed DRL, Gorila [69], is proposed. Gorila utilizes a cluster of machines to achieve an order of magnitude of reduction in training time than single-machine training. To the best of our knowledge, this is the enlightenment work that many methods, such as Ape-X [67], IMPALA [61], and R2D2 [68], spring up in this direction. From the evolution of related works, we can see that cluster computing is roughly a standard mode of accelerating training for DRL.

5.2 Single Machine Parallelism

Other than cluster computing that uses multiple machines, single machine parallelism utilizes multiprocessors or multi-core CPUs in a single machine to increase the computing ability. The rationale for applying single machine parallelism is that distributed clusters may not be affordable for most researchers, unlike the widely available commodity workstations. Besides, data transfer, model synchronization across machines during training will also incur noticeable overhead. Multiprocessing and multithreading are the key technologies in single machine parallelism. More specifically, multiprocessing refers to operating different processes simultaneously by more than

one CPU processor, while multithreading refers to executing multiple independent threads in parallel by a single CPU, especially the multi-core CPU.

The representative method is A3C [70]. In A3C, many actor-learner threads are launched in parallel, and each thread performs a training procedure separately, including environment interaction, experience collection, and model update. In this way, the speed of data generation and the sample efficiency can be improved significantly. The results in [70] showed that A3C using 16 CPU cores surpasses DQN variants using Nvidia K40 GPU in half of the training time and achieves comparative performance to Gorila which uses 100 machines.

Petrenko et al. present a high-throughput training system named SampleFactory [60], which is designed based on a single machine with a multi-core CPU and a GPU. Though optimizing the efficiency and resource utilization in a single machine setting, Sample Factory can achieve throughout as 130 K FPS (Frame per Second) and four times speedup over the state-of-the-art baseline SEED_RL [77] with a workstation-level PC. One of the significant contributions of SampleFactory is that it allows large-scale DRL experiments accessible to a wider community by reducing the computational requirements.

A combination of cluster computing and single machine parallelism is quite common in literature. Fiber [122] extends multiprocessing from one machine to distributed environments. AlphaGo [2] uses asynchronous multi-threads to improve the performance in large-scale Monte Carlo tree search. IMPALA [61] provides single machine settings as well as distributed settings to accelerate large-scale DRL training. The benefit of single machine parallelism over cluster computing is removing the communication overhead of sending data across machines by keeping all the computing in a single machine. Note that the latency of synchronizing the model may have a noticeable impact on the stability of learning, especially for on-policy learning.

5.3 Specialized Hardware Architectures

Since there are lots of batch processing operations (e.g., matrix multiplication) in neural network training and inference, the specialized computation hardware architectures with large throughputs such as **Graphics Process Units** (**GPUs**) , **Field Programmable Gate Arrays** (**FPGAs**) , and **Tensor Process Unit** (**TPUs**) are preferable. With massive **arithmetic and logic units** (**ALUs**) and good programmability, GPUs are utilized to accelerate in a wide range of applications that go well beyond traditional graphics processing. Comparably, FPGA supports customized operations for a specific application by user-programmable interconnects. Although FPGAs might not be superior to GPUs in computation efficiency, they are preponderant in reducing energy consumption. In addition, TPUs are custom-developed **application-specific integrated circuits** (**ASICs**) for machine learning workloads and achieve high throughput and low energy consumption for matrix computations. Overall, all these hardware architectures can exploit the inherent computing parallelism to achieve speedup on data-intensive computing tasks.

In recent years, Nvidia has been devoting itself to utilizing GPUs in distributed DRL. Mohammad et al. propose GA3C [110], an architecture based on A3C [70] with highlighted on GPU utilization to augment the training speed. GA3C employs trainer threads to collect batches of data and submit them to a GPU for exploiting the GPU's computational capabilities better. Rather than utilizing a single GPU, a multi-GPU RL framework has also been proposed [96], in which 32 GPUs in maximum are reported in the experiments. Based on that, hundreds to thousands of locomotion robots are parallelized to accelerate the training. Lasse et al. propose IMPALA [61] to solve large-scale, multi-task RL problems with a multi-GPU system (NVIDIA DGX-1), which contains 8 NVIDIA Tesla V100 GPUs.

FPGAs are utilized in tabular RL for acceleration in the early days [123, 124]. As the neural network has been induced in RL, more room for improvement by FPGAs is created. Many works are

proposed to use FPGAs in accelerating backpropagation in DRL. Su et al. [115] propose an FPGA acceleration system for DQN, which allows dynamically reconstructing the network to achieve better learning results. Their experiment results show that the proposed system can achieve up to 346 times and 77 times speedup compared to GPU implementation and CPU implementation, respectively. Cho et al. propose a FPGA-based A3C Deep RL platform named FA3C [118]. In FA3C, simple and generic processing elements are designed for all types of DNN layers. **Compute unit** (**CU**) pairs are also proposed for inference and training tasks separately. Experiment results show that FA3C achieves 27.9% higher IPS values (the number of inferences processed per second) and 1.62 times better energy efficiency than GPU implementation (Nvidia Tesla P100).

TPUs are designed specifically for machine learning workloads and used frequently in the latest Google findings in the DRL domains such as AlphaZero [3], AlphaStar [4], GATO [121], and so on. In terms of distributed training platforms, Google research teams propose a scalable RL framework called SEED_RL [77], in which multi-TPUs architecture is utilized and the learning speed is significantly improved compared to the baseline based on multi-GPUs. For example, an 11 times speedup is achieved by SEED_RL over a strong baseline IMPALA on DeepMind tasks.

Although these specialized hardware architectures have brought a huge speedup, CPUs also play an essential role in DRL training (e.g., environment interaction, task scheduling, and parameter sharing). A heterogeneous architecture of CPUs and specialized hardware is a promising endeavor in this area. Combining CPUs and GPUs has become the popular way of saturating the computation power of GPUs in recent years, and many works have been proposed to utilize CPU-GPU heterogeneous architectures [61, 75, 110, 114]. Other than this, Meng et al. propose a CPU-FPGA heterogeneous platform for accelerating Proximal Policy Optimization [119]. Rather than only focusing on specific RL algorithms, Meng et al., further propose a more generic CPU-FPGA accelerator using on-chip replay management for widely used RL algorithms including DQN and DDPG [120]. By allocating the workloads to CPU and FPGA sophisticatedly, CPU-FPGA heterogeneous method achieves significant improvement in overall throughput.

5.4 Discussion

Table 1 compares the computing parallelism types utilized in current parallel and distributed DRL implementations. We can see in the table that all of these parallelism types are widely used, and a combination of different computing parallelism techniques is popular. More specifically, all these three types of parallelism are reported in the latest research such as Ray RLlib [105], SEED_RL [77], Dactyl [6], AlphaZero [3], GATO [121], and so on. As parallelism in DRL becomes more and more ambitious, achievements in training efficiency in this area have been proven. The state-of-the-art solution only took 3.6 mins [105] to train Atari 2600 games. In comparison, it took 15 days to train a single Atari game several years ago, which at that time was a significant breakthrough in human history [1].

Besides, with the parallelization of computing resources, computing task scheduling plays an important role in enhancing the efficiency of distributed DRL systems. The computing workloads involved in distributed DRL training are diverse and heterogeneous, including environment interaction, network interference, gradient backpropagation, and model synchronization, which are handled by workers such as Actors, Learners, or the Parameter Server. Hence, it is essential to effectively schedule these computing tasks across processors even machines with varying resource configurations, thereby minimizing the overall training time. There are many scheduling strategies have been proposed in current distributed DRL systems.

— Load balancing. This strategy enables load-aware scheduling that assigns computing tasks to distributed hardware, considering factors such as resource contention and input locality.

Table 1. Comparison of Computing Parallelism Types in Parallel and Distributed DRL Implementations

Methods	Computing parallelism Types					Implementation Details	Major Results
	CC	MP/MT	GPU	FPGA	TPU		
Gorila [69]	✓					31 machines	10× speedup over GPU implementation
Ape-X [67]	✓		✓			360 CPU cores and 1 P100 GPU	4× median scores over Gorila
R2D2 [68]	✓		✓			256 actors and 1 GPU	4× median scores over Ape-X
IMPALA [61]	✓	✓	✓			500 CPU cores and 8 P100 GPUs	250K FPS and multi-task setting
Ray RLlib [105]	✓	✓	✓			8,192 CPU cores on EC2	completes training Mojoco in 3.7mins
ARS [106]	✓					48 CPU cores on EC2	15× speedup over ES-based method [97]
A3C [70]		✓				16 CPU cores	2× speedup over K40 GPU implementation
Reactor [85]	✓	✓				20 CPU cores	4× speedup over A3C
DBA3C [107]	✓	✓				64 nodes with 768 CPU cores	completes training Atrai 2600 in 21 mins
DPPO [108]		✓				64 actors	>20× speedup over A3C
D4PG [109]		✓				64 CPU cores	4× higher return than PPO
SampleFactory [60]		✓	✓			36 CPU cores and a 2080Ti GPU	4× speedup over SEED_RL
GA3C [110]		✓	✓			16 CPU cores and 1 Titan X GPU	45× speedup over A3C
PAAC [111]		✓	✓			4 CPU cores and a GTX 980 Ti GPU	>6× speedup over Gorila
rlpyt [71] [75]		✓	✓			8 P100 GPUs and 40 CPU cores	6× speedup using 8 GPUs relative to 1 GPU
Dactyl [6]	✓	✓	✓			384 nodes (6144 cores and 8 GPUs)	5.5× speedup over implementation with 1 GPU and 768 CPU cores
DD-PPO [72]		✓	✓			256 V100 GPUs	196× speed up over 1 V100 GPU
MSRL [112]		✓	✓			64 GPUs	3× speedup over Ray RLlib
SRL [78]	✓		✓			15K CPU cores and 32 A100 GPUs	5× speedup over OpenAI Rapid [113]
SpeedyZero [114]	✓		✓			192 CPU cores and 20 A100 GPUs	mastering Atari benchmark within 35 minutes using only 300k samples.
NNQL [115]				✓		Arria 10 AX066 FPGA	346× speedup over GTX 760 GPU
TRPO_FPGA [116]				✓		Intel Stratix-V FPGA	19.29× speedup over i7 CPU
DDPG_FPGA [117]				✓		Intel Stratix-V FPGA	4.53× speedup over i7-6700 CPU core
FA3C [118]		✓		✓		Xilinx VCU1525 VU9P FPGA	27.9% better than Tesla P100
PPO_FPGA [119]		✓		✓		Xilinx Alveo U200	27.5× speedup against Titan Xp GPU
On-chip replay [120]		✓		✓		Xilinx Alveo U200 acceler	4.3× higher IPS over GTX 3090 GPU
AlphaZero [3]	✓	✓			✓	5000 TPUs v1 and 64 TPUs v2 cores	defeats world-champion program by training within 24 hours
AlphaStar [4]	✓	✓			✓	3,072 TPU v3 and 50,400 CPU cores	achieves above 99.8% of ranked human players by training in 44 days
OpenAI Five [113]	✓	✓			✓	1,536 GPUs and 172,800 CPU cores	defeats Dota 2 world champion (Team OG) by training in 10 months
GATO [121]	✓	✓			✓	256 TPU v3 cores	handles 604 distinct tasks with a single network
SEED_RL [77]	✓	✓			✓	520 CPU and 8 TPU v3 cores	11× faster than the IMPALA with a P100 GPU

CC: Cluster Computing; MP/MT: Multiprocessing or Multithreading; Statistics are collected from the corresponding articles.

Ray [125] utilizes fine-grained and coarse-grained load balancing methods for stateless and stateful computations, respectively. rlpyt [75] forms two alternating groups of simulation processes, and schedules GPU to serve each group in turn to keep utilization high. Meng et al. [119] propose inter-load balancing method for two CUs involved in training value and policy networks. Li et al. [79] propose to explicitly control the frequencies for parallel workers to achieve load balance.

— Resource dynamic adjusting. This strategy dynamically scales the resources for workers to accelerate DRL training with minimum costs. MINIONSRL [126] leverages a scheduler that

dynamically adjusts the number of Actor workers to optimize the DRL training with minimal training time and costs. Similarly, Meng et al. design a scheduling mechanism that reallocates CPU threads to processing a sub-batch of training for the Learner in heterogeneous computing environments.

— Computation and communication overlapping. In distributed DRL training, computing tasks can be blocked by communication tasks due to their execution dependencies. For example, gradient aggregation has to wait for the completion of gradient transfer for parallel workers. This strategy schedules the computing and communication tasks to reduce the waiting time, thereby improving the computation utilization. PEARL [127], an open-source distributed DRL framework, designs a Learner module that enables scheduling to overlay the communication and computation. Mei et al. [114] propose an optimized data transfer scheduling technique that overlaps computation in distributed DRL training.

— Preemptive scheduling. This strategy proactively terminates lagging tasks, as each worker must wait for all parallel workers to complete during synchronous training. Wijmans et al. propose a straggler preemption mechanism in DDPPO [72], where the slow-running worker is preempted once a certain percentage of the other parallel workers are completed collecting their experience data in one iteration. Similarly, PyTorchRL [128] employs a comparable preemption mechanism within its distributed DRL training framework.

6 Distributed Synchronization Mechanisms

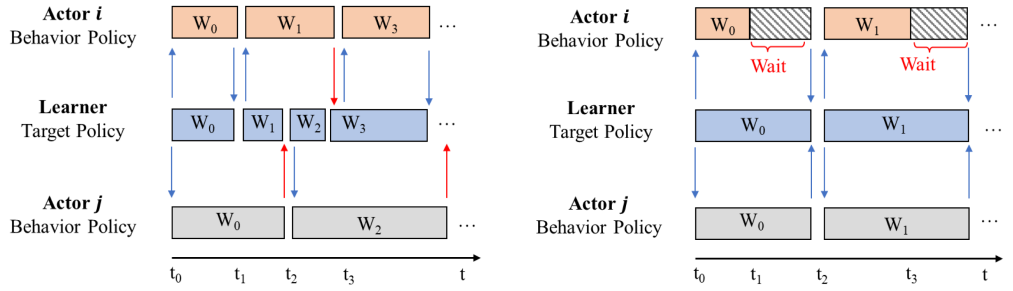
The dominant solution[1] for training acceleration in distributed DRL is employing a number of workers to cooperatively train the model based on ***distributed stochastic gradient descent (DSGD)*** [27, 48]. This is based on the data parallelism introduced in Section 2. During DSGD training, each parallel work holds a copy of the model, performs training based on a subset of environmental experiences, and then aggregates the update to the target model cooperatively. It is noted that distributed synchronization among workers is vital to the training efficiency, especially in the heterogeneous environment where the parallel workers may process at different speeds. Variants of synchronization mechanisms are systemically studied in DL area, i.e., **Bulk Synchronous Parallel** (**BSP**) [129], **Asynchronous Parallel** (**ASP**) [130], and **Stale Synchronous Parallel** (**SSP**) [131]. These mechanisms lay a good technical foundation for distributed DRL.

However, since importing environment interaction and dividing the parallel workers into different roles such as Actors and Learners, DRL may face new difficulties while applying distributed synchronization approaches. The model holding in Actors for the behavior policy may lag behind the model in Learners for the target policy after several iterations in parallel settings. This makes the behavior policy different from the target policy gradually. It is noted that there are off-policy and on-policy schemes for DRL algorithms (see Section 2). According to the training schemes and the way of synchronization among workers, there are mainly two types of distributed synchronization mechanisms in current distributed DRL solutions: asynchronous off-policy training and synchronous on-policy training.

6.1 Asynchronous Off-policy Training

Asynchronous off-policy training is the distributed synchronization mechanism that was initially applied in distributed DRL, and it has been widely used later on, e.g., Gorila [69], APE-X [67], R2D2 [68], and so on. In the asynchronous training, the parallel workers operate independently. As shown in Figure 6, each worker performs model update at its own pace without waiting for other workers. This will naturally lead to differences between the behavior policy model for Actors and

[1]Another promising training solution based on evolutionary learning will be discussed in Section 7.



(a) Asynchronous Off-policy Training (b) Synchronous On-policy Training

Fig. 6. Comparison between different synchronization mechanisms in distributed DRL. The rectangular bar represents the corresponding training step for the Actor and the Learner. Arrows represent data transmission for the model parameter update or the rollout data.

the target policy model for Learners. We can see in Figure 6(a), at time t_1, the behavior policy with parameter W_0 for Actor j is different from the target policy with parameter W_1 for the Learner. The same phenomenon can also be seen at time t_2 , when the behavior policy with parameter W_1 for Actor i is different from the target policy with parameter W_2 for the Learner. Therefore, this type of synchronization mechanisms is mainly used for off-policy algorithms which do not require the consistency between the behavior policy and the target model while learning. DQN is a classic off-policy algorithm, and many asynchronous distributed DRL methods such as Gorila, APE-X, and R2D2 are based on DQN. More specifically, asynchronous off-policy training is normally based on a centralized architecture and there is a Parameter Sever maintaining the global model. Each Actor pushes its interaction training data to the Replay Memory and pulls the up-to-date parameters from the Parameter Server asynchronously. The Learner updates the global model based on the training data in the Replay Memory. In this way, every worker tries to optimize the global model by contributing its efforts based on the local copy of the model, thereby accelerating the model training.

However, it is known that off-policy algorithms suffer from the stability issue [37]. That is mainly because the distribution under the target policy shifts from that under the training data collected by the behavior policy, named *policy-lag* [61]. To relieve this problem, there are many off-policy correction methods have been proposed in distributed DRL. Espeholt et al. propose *V-trace* correction methods in IMPALA [61]. In this method, *V-trace targets* are computed based on trajectories collected by the behavior policy and the current target value function under the target policy, where two truncated **importance sampling** (**IS**) weights are imported to improve convergence. Babaeizadeh et al. propose ϵ-correction in GA3C [110], which adds a small constant ϵ during gradient estimation. This prevents the log value of the sampled action probability from becoming very small and leading to numerical instabilities.

Furthermore, the nature of the asynchronous update will also cause *stale update*, which occurs when a worker updates the target model using training data generated by the obsolete model [129]. As shown in Figure 6(a), the Learner, Actor i, and Actor j at time t_0 hold the model denoted as W_0. Actor i at time t_1 pushes the training data to create a new model denoted as W_1. Actor j at time t_2 pushes the training data to create a new model denoted as W_2. However, at time t_2, Worker j's model used in training (i.e., W_0) is stale compared to the target model (i.e., W_1). The stale update, shown as red arrows in Figure 6(a), may slow down the convergence in training [70, 132]. Distributed *model-based* DRL methods also suffer from this issue. For example, *reanalyze staleness* is incurred in SpeedyZero [114]. This is because the parallel trainers commonly receive batches that have been reanalyzed by an outdated version of the model, rather than the latest target model.

6.2 Synchronous On-policy Training

In order to mitigate the stale update issue mentioned above, the synchronous on-policy training method lets the parallel workers wait for the completion of gradient computation for all the workers, and then performs the model update synchronously. In this way, the models contained in parallel workers are consistent. This consistency among workers ensures the on-policy property of the DRL algorithm that the behavior policy and the target policy are the same during training. Current synchronous on-policy training solutions in distributed DRL mainly use two ways to synchronize the model: centralized and decentralized manners. For the former one such as PAAC [133] and DBA3C [107], every worker transmits its gradients to the Parameter Server for model optimization. Then the updated model is copied to every worker. With respect to the decentralized manner such as DDPPO [72], all worker's gradients are shuffled among them. Then, the model in each worker is updated based on the aggregated gradients. In both ways, the parallel workers update the model synchronously and consistently. It is clear in Figure 6(b) that there is no stale update that each worker holds the consistent model in each training iteration.

However, it may incur the synchronization barrier, where the fast-running worker keeps idle and waits for the slow-running one in each iteration, as shown in Figure 6(b). At time t_1, after Actor i completes its training iteration, it waits for Actor j to complete. This will deteriorate the utilization of computing resources in the cluster [72]. It is known that heterogeneity is the marked characteristic in the real cluster environment [134], where the case of processing at different speeds for different workers is quite common. Besides, the gradient aggregation induces burst traffic while transmitting the gradients for all workers in a short duration. This may cause congestion and then reduce the utilization of communication resources.

6.3 Discussion

Although asynchronous off-policy training methods have been shown higher resource utilizations than synchronous counterparts, they usually suffer from stability issues and converge to poorer results [129, 135]. Besides, recent works have demonstrated experimentally that synchronous on-policy training performs better than asynchronous off-policy training when using a large scale of distributed nodes. Adamski et al. [107] witnessed that synchronous on-policy training outperformed asynchronous off-policy training in the experiments of training Atari games using 64 nodes.

To alleviate the synchronization barrier in synchronous training, stale-synchronous training introduces flexibility in the strict synchronization timing, allowing faster workers to proceed to the next iteration under the condition of bounded staleness (i.e., a concept of the maximum obsolete age between the slowest and other workers). Once the staleness bounded is reached, a synchronous model update is enforced. In this way, a better tradeoff between model consistency and resource utilization can be achieved. To the best of our knowledge, while stale-synchronous training achieves decent results in DL area [136, 137], it is not yet widely adopted in distributed DRL currently. We believe stale-synchronous training holds great potential value in distributed DRL. One possible way is to enforce a synchronization according to the divergence between the target policy and the behavior policy.

Moreover, beyond vanilla on-policy and off-policy training, combining on-policy and off-policy in distributed DRL has attracted much attention in recent years. Schmitt et al. [138] propose to mix the off-policy replay experience with on-policy data and introduce a trust region algorithm that efficiently mitigates bias and enables efficient learning in distributed settings. Results show that this method outperforms IMPALA [61] based on V-trace IS. Other than this, Borges et al. [139] propose to combine the off-policy targets with the on-policy targets in the distributed model-based DRL system named MuZero, improving the convergence speed and rewards.

7 Deep Evolutionary RL

As mentioned above the dominant solution for training the neural network in distributed DRL is DSGD based on backpropagation of gradients. On the other hand, evolutionary computation, an approach inspired by the process of natural selection, finds its great potential in solving DRL problems. Many evolution-based training methods for distributed DRL have been proposed in recent years and have demonstrated impressive performance [8, 66, 97, 140–142]. This field of research is also named Deep Evolutionary RL [8] and neuroevolution [142] in the literature. In the interest of brevity, evolution-based training methods directly perform searches in the parameter space by evolving a population of candidate solutions over many generations, without the need for backpropagating and aggregating gradients. As a result, evolution-based training methods enable massive parallelization with a low bandwidth requirement.

7.1 Training Acceleration Based on Evolution Strategies (ES)

ES is one of the major branches of evolutionary computation, which iteratively updates a search distribution by using an estimated gradient on the parameter spaces [144]. There are many proposals that utilize ES algorithms to solve DRL problem [97, 141, 145]. One representative research is [97] proposed by Salimans et al. In this method, a population of parameters (*genotypes*) and their objective function values (*fitness*) for the neural network are maintained for every iteration (*generation*). The parameters with the greatest fitness are selected to form the population for the next generation. This iteration is ended while the optimization objective is achieved. Let $F(\theta)$ represent the objective function parameterized by θ, and p_μ denotes the distribution (parameterized by μ) that the population follows. The goal of the method is to maximize the expectation value $\mathbb{E}_{\theta\sim p_\mu} F(\theta)$ over the population by searching for μ with stochastic gradient ascent. In terms of solving the DRL problem, the objective function $F(\theta)$ is the return after a sequence of actions $\mathbf{a}$ is taken, denoted by $R(\mathbf{a}(\theta))$, where the actions are determined by a policy $\mathbf{a} = \pi(s|\theta)$. The method supposes the distribution p_μ as isotropic multivariate Gaussian with mean μ and fixed covariance $\sigma^2 I$. Then, $\mathbb{E}_{\theta\sim p_\mu} F(\theta)$ can be written in the form of a Gaussian-blurred version of the original objective, that is,

$$\mathbb{E}_{\theta\sim p_\mu} = \mathbb{E}_{\epsilon\sim\mathcal{N}(0,I)} F(\theta + \sigma\epsilon). \tag{1}$$

Hence, the ES method performs the optimization by stochastic gradient ascent over θ directly with the following estimator:

$$\nabla_\theta E_{\epsilon\sim\mathcal{N}(0,I)} F(\theta + \sigma\epsilon) = \frac{1}{\sigma} E_{\epsilon\sim\mathcal{N}(0,I)} F(\theta + \sigma\epsilon)\epsilon. \tag{2}$$

Note that the ES method does not calculate gradients analytically but estimates the gradients of the objection function over parameters accordingly. By exploring in the parameter space instead of action space compared to policy gradient methods, the ES algorithms can be considered as a black box optimization that is invariant to action frequency and delayed rewards. This is thus better suited for problems with a long time horizon. Salimans et al. [97] show that their algorithm achieves comparable results in continuous robotic control problems and video games.

Another surprising advantage of the method in [97] is the massive parallelization ability. This method incurs relatively low communication overhead when parallelized across many workers for the following reasons: First, since the operation in each iteration is conducted over the entire episode, the frequency of communication between workers is significantly reduced. Second, the information that needs to be transmitted between workers is limited to the return of an episode. This requires less bandwidth compared to distributed gradient-based methods, where the transfer involves gradients of the parameter vector or the parameter vector itself. Third, the elimination

of value function approximations helps to reduce communication overhead, as there is no requirement for the additional gradient synchronization needed to learn the value function. Salimans et al. [97] have deployed their algorithm over 80 machines with 1,440 CPU cores, which achieves training time reduction by two orders of magnitude compared to single machine deployment.

7.2 Training Acceleration Based on Genetic Algorithms (GA)

GA is another branch of evolutionary computation, which is based on the theory about the genetic structure and behavior of chromosomes [146]. Different from the ES-based method [97] which is still gradient-based optimization, GA-based methods are gradient-free. Such et al. [66] propose a GA-based method called Deep GA for solving DRL problem. Deep GA is based on a genetic algorithm which normally consists of three main operations: selection, mutation, crossover. In terms of selection, truncation selection is performed, which selects the top T individuals as the parents of the next generation. The mutation is performed by adding Gaussian noise to the parameter vector shown as

$$\theta^n = \psi(\theta^{(n-1)}, \tau_n) = \theta^{(n-1)} + \sigma\epsilon(\tau_n), \tag{3}$$

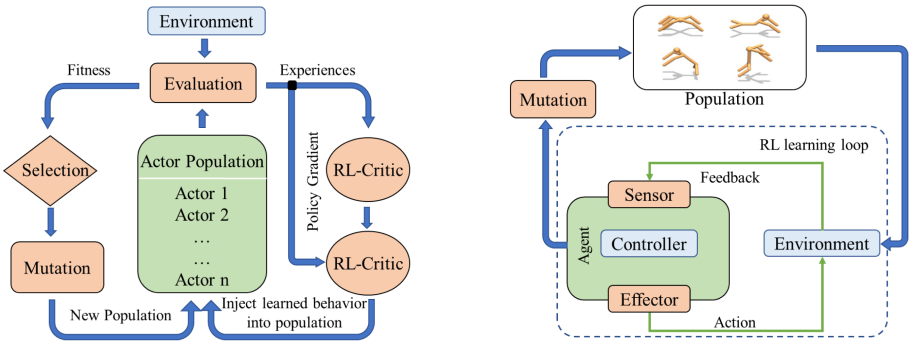
where θ^n is the next generation of $\theta^{(n-1)}$, $\psi(\theta^{(n-1)}, \tau_n)$ is a mutation function and τ_n is a list of mutation seeds for θ^n. $\epsilon(\tau_n)$ follows Gaussian distribution $\mathcal{N}(0, I)$. With respect to crossover, Deep GA does not include it for simplicity. Besides, in order to scale well in the distributed setting, Deep GA leverages a compact encoding technique that uses an initialization seed plus a list of random seeds to reconstruct the parameter vector. The experiment results show that Deep GA can outperform ES [97], A3C, and DQN on average. The results also reveal that Deep GA is superior to the gradient-based method in solving local optima problems by jumping across them in the parameter space.

Unlike only evolving weights of the neural networks in [66], Stanley et al. propose a GA-based method called NEAT [147, 148], which evolves network topologies along with weights to enhance the learning efficiency. NEAT has been successfully applied to policy optimization and outperforms baseline with fixed-topology on an RL benchmark. Although NEAT and HyperNEAT [149] represent early works for the topology evolution of small networks, there are many recent works targeting deep (large-scale) neural networks [8, 150–153], including evolving the hyperparameters [154, 155]. It is known that the architecture and hyperparameters of neural networks highly impact the overall performance. Success in the applications of deep (reinforcement) learning often requires fine-tuning of these building blocks of networks manually in practice. Fortunately, with the help of these solutions, the issue of manually designing a neural network for every application can be relieved.

7.3 Hybridization of Evolutionary Computation and Backpropagation

The aforementioned methods mainly follow the path of performing a direct search in the parameter (or encodings) space based on the evolutionary computation, thereby obtaining the optimized neural networks that can serve as the policy in DRL problem. While evolutionary computation has merits of diverse exploration and massive parallelism based on a population-based scheme, it suffers from high sample complexity, especially for problems with large numbers of parameters. Meanwhile, DRL methods leverage powerful backpropagation approaches to reinforce profitable actions into weight parameters. Hence, a hybrid scheme that combines the strengths of evolutionary computation and backpropagation has great potential value and triggers wide concerns.

Khadka et al. [143] propose an evolution-guided policy gradient method in DRL. This method incorporates evolutionary algorithms to generate diverse experiences and utilizes backpropagation



(a) Evolution-guided Policy Gradient Method[143] (b) Deep Evolutionary Reinforcement Learning[8]

Fig. 7. Architectures of current methods based on a combination of learning and evolution.

in DRL to learn from them. More specifically, this method combines a genetic algorithm with DDPG algorithm, as shown in Figure 7(a). A population of actor networks is maintained to interact with the environment and generate diverse experiences in parallel. The next generation of actors is created by selection, mutation, and crossover operators. Besides, the critic network and the actor network are trained by using backpropagation based on samples from the reply buffer. Then, the updated weights of the actor network are synchronized to the population of the actor networks. In this way, this method injects the learned behaviors based on DRL into the evolving population. Experiments based on continuous control benchmarks show that this hybrid method outperforms prior DRL and evolutionary algorithms.

Gangwani et al. [156] propose a hybrid algorithm named **Genetic Policy Optimization** (**GPO**) for sample-efficient deep policy optimization. Rather than perform crossover in the parameter space in existing methods [143, 147, 149], GPO does crossover in the state visitation space with the goal of cloning the behaviors of parents. Research showed that the crossover way of exchanging the parameters straightforwardly often leads to hierarchical relationship destruction and a loss of functionality [147]. Besides, GPO utilizes the DRL algorithm (PPO) based on backpropagation instead of random permutation to mutate the weights of the actor networks. Experiments on Mojoco benchmark locomotion tasks demonstrate that GPO outperforms PPO and A2C [157] algorithms and achieves comparable or higher sample efficiency.

7.4 Discussion

Neuroevolution enables useful capabilities that are basically unavailable for traditional DRL approaches, including evolving building blocks of neural networks such as weights, topologies, and hyperparameters, facilitating diverse exploration and massive parallelization while maintaining the evolving population of networks. This also opens the door to incorporating learning and evolution to create sophisticated intelligence for solving high-dimensional decision-making problems. Other than the approaches above, there are still many interesting topics in this exciting area. Some examples incorporate quality diversity [158, 159], novelty search [160, 161] and adaptive noise [162, 163] to improve exploration. Many works focus on **Neural Architecture Search** (**NAS**) by utilizing indirect encoding [164–166], swarm intelligence algorithms [167–169], random search [170–172], sequential model-based optimization [173, 174], and so on. While NAS is still in the initial research stage, breakthroughs have been made in many fields including image classification [175, 176], object detection [177, 178], machine translation [179, 180], and multi-task learning [181, 182].

Besides, with the increasing capacities of computing resources that are easy to obtain, neuroevolution has further demonstrated its great potential in solving complex problems. Li et al. utilize deep evolutionary RL to evolve diverse agent morphologies to handle locomotion and manipulation tasks in complex environments [8]. Its architecture is shown in Figure 7(b). This method employs 1,152 CPUs to evolve ten generations of populations and train four thousand agent morphologies, with five million environment interactions for each morphology. Real et al. propose to automatically discover neural network models for CIFAR-10 and CIFAR-100 datasets by evolving 1,000 population of models with 250 parallel workers [151]. Asseman et al. study to use FPGA acceleration on Deep GA [66] method with a distributed system of 432 Xilinx FPGAs [183]. In their experiment, 832 instances in parallel can be run, and 1.2 million FPS of the aggregation rate can be achieved. Compared to the baseline method [66], this method achieves twice the speedup and high game scores in most cases.

8 Open-source Libraries and Platforms

In AI area that is flourishing, it is undoubted that developing and evaluating innovations in a rapid manner is essential in academia as well as the industrial community. This motivates researchers to develop tools, such as libraries and platforms, to facilitate DRL algorithm development in the field. In recent years, many libraries and platforms have been proposed in DRL areas, e.g., OpenAI baselines,[2] Keras-RL,[3] MushroomRL,[4] and so on. Some of these are scalable and user-friendly that algorithmic components (functions, neural networks, environments, etc.) can be flexibly reused to compose customized learning agents. However, many of them are designed to be run on a single process or a single machine. Parallel and distributed execution at scale are not considered. Parallel and distributed programming have high professional restrictions that require complex operations, which are not friendly to developers who major in learning algorithm design. Therefore, there is a crucial demand to encapsulate parallelism primitives in libraries and platforms.

We review and compare current open-source libraries or platforms that support parallel and distributed DRL according to the following criteria.

- Baseline Algorithms. They should provide rich choices of existing DRL algorithms, which facilitates rapid development and comparisons for users.
- Environment Integration. Simulation interaction is a basic element in DRL training. Hence, integrating simulation environments in training libraries and platforms plays an important role in supporting easy verification as well as applications.
- Parallel and Distributed Features. The key parallel and distributed techniques and capabilities offered to enhance the DRL training efficiency are compared.

It is noted that we do not compare the performance of existing libraries and platforms experimentally, which is out of the scope of this survey article. Table 2 provides an overview of the open-source libraries or platforms that support parallel and distributed DRL. Statistics are collected from the Github links, documents, and articles of corresponding codebases in Aug. 2024.

Most of these libraries and platforms provide rich choices of baseline algorithms and supported environments. More specifically, RLlib [105], rlpyt [71], ACME [187], and TorchRL [190] include value-based and policy gradient families of DRL algorithms, and provide baselines of the SOTA parallel and distributed DRL methods such as IMPALA [61] and Ape-X [67]. In comparison, the open-source version of SampleFactory [60] and SRL [78] only implement one algorithm (Proximal Policy Optimization, PPO). In terms of supported environments, all of the libraries and platforms

[2] https://github.com/openai/baselines
[3] https://github.com/keras-rl/keras-rl
[4] https://github.com/MushroomRL/mushroom-rl

Table 2. Open-source Libraries/Platforms for Parallel and Distributed DRL

Libraries/Platforms	Institutes	Year	Baseline Algorithms	Supported Envs.	Parallel and Distributed Features
TF-Agents [184]	Google	2017	DQN, DDQN, DDPG, TD3, REINFORCE, PPO, SAC, and so on.	Bsuite, DeepMind Control, Gym, MuJoCo, Pybullet	Batch simulation and network forward pass parallelism based on CPUs/TPUs.
Ray RLlib [105]	Berkeley	2017	A2C, A3C, ARS, BC, DDPG, TD3, Rainbow, ES, APE-X, IMPALA, PPO, APPO, DD-PPO, SAC, QMIX, VDN, IQN, MADDPG, MCTS, and so on.	Gym, PettingZoo, Unity3D, Gazebo	Synchronous training with straggler migrations, and various baselines including evolution-based and model-based algorithms
ReAgent [185]	Facebook	2018	DQN, Double DQN, Dueling DQN, QR-DQN, DDPG, TD3, SAC, and so on.	Gym	Workflows to perform training with large-scale data preprocessing, feature transformation, distributed training, and so on.
SURREAL [7]	Standford	2018	DDPG and PPO	Gym, Deepmind Control, Surreal Robotics, MuJoCo	Asynchronous training and cluster automation setup on major cloud providers, e.g., AWS and Azure.
PARL [186]	Baidu	2019	DQN, ES, DDPG, PPO, IMPALA, A2C, TD3, SAC, MADDPG, CQL, QMIX, and so on.	Gym	Parallelization of training with thousands of CPUs and multi-GPUs based on a centralized architecture.
rlpyt [71]	Berkeley	2019	A2C, PPO, DQN, DQN variants, Rainbow, R2D2, Ape-X, DDPG, TD3, SAC, and so on.	Atari, Gym	Synchronous and asynchronous sampling and optimization.
SEED_RL [77]	Google	2020	IMPALA, R2D2, SAC, Vanilla PG, PPO, AWR, V-MPO, and so on.	Atari, DeepMind Control, Google Football, MuJoco	Centralized inference and a fast communication layer based on gRPC.
ACME [187]	DeepMind	2020	D4PG, TD3, SAC, MPO, PPO, DMPO, MO-MPO, DQN, IMPALA, R2D2, MCTS, and so on.	DeepMind Control, Gym	Asynchronous training with a learner process and many distributed actor processes and low-level storage system *Reverb* for experience replay.
Fiber [122]	Uber	2020	A3C, PPO, ES, and so on.	ALE, Gym, MuJoCo	Standard multiprocessing API and online migration from multiprocessing on one machine to cluster computing across multiple machines.
SampleFactory [60]	Intel Lab	2020	PPO	MuJoCo, Atari, VizDoom, DeepMind Lab, Megaverse, Envpool, IsaacGym, Brax, Quad-Swarm-RL, Hugging Face Hub	Asynchronous training with off-policy corrections on a single-machine setting.
ChainerRL [188]	Preferred Networks	2021	DQN, IQN, Rainbow, REINFORCE, A2C, ACER, PCL, DDPG, TRPO, PPO, TD3, SAC, and so on.	Gym, MuJoCo	Synchronous and asynchronous training.
Tianshou [189]	Tsinghua	2022	REINFORCE, A2C, TRPO, PPO, DDPG, TD3, SAC, DQN, Rainbow, IQN, BCQ, CQL, CRR, PER, PSRL, and so on.	Gym, PettingZoo	Synchronous and asynchronous training, and standardization support of the training process.
TorchRL [190]	UPF	2023	A2C, PPO, DDQN, SAC, REDQ, Dreamer, Decision transformers, RLHF, APPO, DPPO, DDPPO, IMPALA, Ape-X, and so on.	Gym, DeepMind Control	Synchronous and asynchronous training, and high modularity that allows flexible composability for DRL training.
MindSpore [112]	ICL, Huawei	2023	DQN, PPO, A2C, A3C, DDPG, MAPPO, MADDPG TD3, SAC, Double DQN, and so on.	Gym, MuJoCo, MPE, SMAC, DMC, PettingZoo, D4RL	Synchronous and asynchronous training, and flexible parallelization using fragmented dataflow graph (FDG).
SRL [78]	Tsinghua	2024	PPO	Gym, Atari, Google Football, MuJoCo, Hide and Seek, SMAC	Synchronous training and data batch pre-fetching.
PEARL [127]	Meta Research	2024	DQN, DDPG	Gym	Portable implementations across heterogeneous platforms including FPGA and Optimization on DRL-specific runtime scheduling.

Statistics are collected from the Github links, documents, and articles of corresponding codebases in Aug. 2024.

integrate OpenAI's Gym and support Gym wrapper API. This provides a lot of convenience for users to access to environments. There are nine codebases that support more than three types of environments, e.g., SampleFactory [60] and MindSpore [112] provide 10 and 7 types of environments, respectively.

Besides, we compare the parallel and distributed features such as parallel training methods and communication mechanisms for different libraries and platforms. RLlib [105] and rlpyt [71] offer training options for synchronous or asynchronous optimization. SEED_RL [77] encapsulates a fast communication layer based on streaming RPCs to mitigate the overhead of remote calls. Fiber [122] provides standard multiprocessing API and supports migration from multiprocessing on one machine to cluster computing across multiple machines on the fly. Besides, some of them [7, 105] provide demonstrations or even automation cluster setups on major cloud providers, e.g., AWS, Azure, and Google Cloud. This is very useful to promote parallel and distributed DRL in practice since most research teams cannot afford to build a cluster of resources.

9 Conclusion and Future Directions

Expanding DRL to a large-scale has become an inevitable trend nowadays. However, the rapid progress in which the field is developing makes it difficult to understand in a systematic manner. In this survey, we have provided a comprehensive literature review on training acceleration for DRL using parallel and distributed computing. In particular, we have analyzed the primary challenges to make DRL training distributed, and demystified the details of the technologies that have been proposed by researchers to address these challenges. This includes the system architectures for distributed DRL, the simulation parallelism to increase sample collection efficiency, the computing parallelism to improve the computational efficiency, the distributed synchronization mechanisms for backpropagation-based training, the deep evolutionary RL for evolution-based training. Furthermore, we have summarized the existing libraries and platforms in view of facilitating the research community in rapid development. This survey tries to enable readers to have a holistic view of distributed DRL and provides a guideline for them to get started quickly in this area. Below, we extrapolate potential directions for future work in this field.

Specialized hardware accelerators. Designing specialized hardware accelerators that are computationally efficient with neural networks becomes an urgent and promising direction. It has been witnessed that many domain-specific architectures have been proposed to accelerate DRL training such as NVIDIA Tensor Core [75], FPGAs [118], and TPUs [77]. Despite the advancements in computation speed, several emerging topics remain in this highly dynamic field of research. One such direction is in-memory computing, which is critical for reducing data movement and minimizing latency for DRL training [120, 191]. Another direction is the exploitation of pipeline parallelism according to the DRL training workloads, which requires innovative hardware designs and algorithms to ensure all the arrays on the hardware are always active during training [192]. Additionally, how to reduce the energy consumption while improving the computational efficiency is also very important. We believe that neuromorphic hardware design is a promising direction, such as spike-based neuromorphic chips [193]. Moreover, heterogeneous architectures that combine the strengths of different hardware platforms in the field of DRL can be further investigated [127].

In-network distributed aggregation mechanisms. Network communication occupies a large part of the execution time in distributed gradient aggregation. To alleviate the communication overhead, an emerging trend is to shift the gradient aggregation process from the worker nodes to the network infrastructure itself, such as at the programmable switches. By doing so, the gradient aggregation is conducted on-the-fly at the granularity of network packets rather than gradient vectors stored within the memory of the worker nodes. This approach has the potential to substantially reduce the end-to-end network latency and the volume of data transferred across the network during distributed training. Li et al. [74] were pioneers in leveraging in-switch computing to accelerate the distributed training of DRL, achieving significant results. We believe designing packet forwarding and computing mechanisms according to the characteristics of DRL workloads will have great potential for training improvement.

Efficient sample exploitation algorithms. A large amount of rollout data is required is one of the known issues of DRL training. One reason is counted for the low sample reuse rate. OpenAI researchers demonstrate that the sample reuse rate is lower than 1 in the asynchronous training of the OpenAI Five [113]. Note that simply collecting more data in parallel without effectively exploiting it does not necessarily translate into improved training outcomes. Therefore, the development of algorithms that can exploit rollout samples more effectively, without compromising the stability of the learning process, is a crucial direction for future research in distributed DRL. To address this challenge, several strategies can be pursued. First, more sophisticated replay mechanisms, including Priority-refresh [114], asynchronous curriculum experience replay [194], and regret minimization replay [195], are promising techniques to better utilize the samples. Besides, driving DRL agents to perform searches sophisticatedly rather than randomly is helping for generating more useful samples. Exploration strategies such as curiosity-driven [196], diversity-driven [197], and novelty search [142] in large state-action spaces are potential directions of importance.

Large language model (LLM)-enhanced DRL. LLMs, equipped with extensive pre-trained knowledge and powerful generalization abilities, are emerging as a promising direction to enhance the performance of DRL, including accelerating the training process. Specifically, in the DRL training paradigm, LLMs can assist DRL agents in reward function design, action selection, and policy evaluation, based on the modeling capability and common-sense pre-trained knowledge. In this way, LLMs not only enable a considerable level of ability at the beginning of the training process, but also facilitate the decision-making during the optimization. Currently, numerous pioneering works [198–200] have been proposed that I am confident will pave the way for robust development in the coming future.

References

[1] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. 2015. Human-level control through deep reinforcement learning. *Nature* 518, 7540 (2015), 529–533.

[2] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. 2016. Mastering the game of Go with deep neural networks and tree search. *Nature* 529, 7587 (2016), 484–489. DOI: http://dx.doi.org/10.1038/nature16961

[3] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. 2018. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science* 362, 6419 (2018), 1140–1144.

[4] Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H. Choi, Richard Powell, Timo Ewalds, Petko Georgiev, and Others. 2019. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature* 575, 7782 (2019), 350–354.

[5] Elia Kaufmann, Leonard Bauersfeld, Antonio Loquercio, Matthias Müller, Vladlen Koltun, and Davide Scaramuzza. 2023. Champion-level drone racing using deep reinforcement learning. *Nature* 620, 7976 (2023), 982–987.

[6] Open AI: Marcin Andrychowicz, Bowen Baker, Maciek Chociej, Rafal Józefowicz, Bob McGrew, Jakub Pachocki, Arthur Petron, Matthias Plappert, Glenn Powell, Alex Ray, Jonas Schneider, Szymon Sidor, Josh Tobin, Peter Welinder, Lilian Weng, and Wojciech Zaremba. 2020. Learning dexterous in-hand manipulation. *International Journal of Robotics Research* 39, 1 (2020), 3–20. DOI: http://dx.doi.org/10.1177/0278364919887447

[7] Linxi Fan, Yuke Zhu, Jiren Zhu, Zihua Liu, Orien Zeng, Anchit Gupta, Joan Creus-Costa, Silvio Savarese, and Li Fei-Fei. 2018. Surreal: Open-source reinforcement learning framework and robot manipulation benchmark. In *Proceedings of the Conference on Robot Learning*. PMLR, 767–782.

[8] Agrim Gupta, Silvio Savarese, Surya Ganguli, and Li Fei-fei. 2021. Embodied intelligence via learning and evolution. *Nature Communications* 12, 5721 (2021), 1–12. DOI: http://dx.doi.org/10.1038/s41467-021-25874-z

[9] Mariya Popova, Olexandr Isayev, and Alexander Tropsha. 2018. Deep reinforcement learning for de novo drug design. *Science Advances* 4, 7 (2018), eaap7885.

[10] Adam Yala, Peter G. Mikhael, Constance Lehman, Gigin Lin, Fredrik Strand, Yung-Liang Wan, Kevin Hughes, Siddharth Satuluru, Thomas Kim, Imon Banerjee, et al. 2022. Optimizing risk-based breast cancer screening policies with reinforcement learning. *Nature Medicine* 28, 1 (2022), 136–143.

[11] Suchi Saria. 2018. Individualized sepsis treatment using reinforcement learning. *Nature Medicine* 24, 11 (2018), 1641–1642.

[12] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. 2018. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

[13] Kai Arulkumaran, Marc Peter Deisenroth, Miles Brundage, and Anil Anthony Bharath. 2017. Deep reinforcement learning: A brief survey. *IEEE Signal Processing Magazine* 34, 6 (2017), 26–38. DOI: http://dx.doi.org/10.1109/MSP.2017.2743240

[14] Yuxi Li. 2017. Deep reinforcement learning: An overview. arXiv:1701.07274. Retrieved from https://arxiv.org/abs/1701.07274

[15] Xu Wang, Sen Wang, Xingxing Liang, Dawei Zhao, Jincai Huang, Xin Xu, Bin Dai, and Qiguang Miao. 2024. Deep reinforcement learning: A survey. *IEEE Transactions on Neural Networks and Learning Systems* 35, 4 (2024), 5064–5078. DOI: http://dx.doi.org/10.1109/TNNLS.2022.3207346

[16] Thomas M. Moerland, Joost Broekens, Aske Plaat, and Catholijn M. Jonker. 2023. Model-based reinforcement learning: A survey. *Foundations and Trends® in Machine Learning* 16, 1 (2023), 1–118.

[17] B. Ravi Kiran, Ibrahim Sobh, Victor Talpaert, Patrick Mannion, Ahmad A. Al Sallab, Senthil Yogamani, and Patrick Perez. 2022. Deep reinforcement learning for autonomous driving: A survey. *IEEE Transactions on Intelligent Transportation Systems* 23, 6 (2022), 4909–4926. DOI: http://dx.doi.org/10.1109/TITS.2021.3054625

[18] Thanh Thi Nguyen and Vijay Janapa Reddi. 2023. Deep reinforcement learning for cyber security. *IEEE Transactions on Neural Networks and Learning Systems Ml* 34, 8 (2023), 3779–3795. DOI: http://dx.doi.org/10.1109/TNNLS.2021.3121870

[19] Nguyen Cong Luong, Dinh Thai Hoang, Shimin Gong, Dusit Niyato, Ping Wang, Ying Chang Liang, and Dong In Kim. 2019. Applications of deep reinforcement learning in communications and networking: A survey. *IEEE Communications Surveys and Tutorials* 21, 4 (2019), 3133–3174. DOI: http://dx.doi.org/10.1109/COMST.2019.2916583

[20] Ammar Haydari and Yasin Yilmaz. 2022. Deep reinforcement learning for intelligent transportation systems: A survey. *IEEE Transactions on Intelligent Transportation Systems* 23, 1 (2022), 11–32. DOI: http://dx.doi.org/10.1109/TITS.2020.3008612

[21] Thanh Thi Nguyen, Ngoc Duy Nguyen, and Saeid Nahavandi. 2020. Deep reinforcement learning for multiagent systems: A review of challenges, solutions, and applications. *IEEE Transactions on Cybernetics* 50, 9 (2020), 3826–3839. DOI: http://dx.doi.org/10.1109/TCYB.2020.2977374

[22] Wuhui Chen, Xiaoyu Qiu, Ting Cai, Hong-Ning Dai, Zibin Zheng, and Yan Zhang. 2021. Deep reinforcement learning for Internet of Things: A comprehensive survey. *IEEE Communications Surveys and Tutorials* 23, 3 (2021), 1659–1692.

[23] Joost Verbraeken, Matthijs Wolting, Jonathan Katzy, Jeroen Kloppenburg, Tim Verbelen, and Jan S. Rellermeyer. 2020. A survey on distributed machine learning. *ACM Computing Surveys* 53, 2 (2020), 1–33.

[24] Diego Peteiro-Barral and Bertha Guijarro-Berdiñas. 2013. A survey of methods for distributed machine learning. *Progress in Artificial Intelligence* 2 (2013), 1–11.

[25] Meng Wang, Weijie Fu, Xiangnan He, Shijie Hao, and Xindong Wu. 2020. A survey on large-scale machine learning. *IEEE Transactions on Knowledge and Data Engineering* 34, 6 (2020), 2574–2594.

[26] Ruben Mayer and Hans Arno Jacobsen. 2020. Scalable deep learning on distributed infrastructures: Challenges, techniques, and tools. *ACM Computing Surveys* 53, 1 (2020), 1–37. DOI: http://dx.doi.org/10.1145/3363554

[27] Tal Ben-Nun and Torsten Hoefler. 2019. Demystifying parallel and distributed deep learning: An in-depth concurrency analysis. *ACM Computing Surveys* 52, 4 (2019), 1–43. DOI: http://dx.doi.org/10.1145/3320060

[28] Karanbir Singh Chahal, Manraj Singh Grover, Kuntal Dey, and Rajiv Ratn Shah. 2020. A Hitchhiker's guide on distributed training of deep neural networks. *Journal of Parallel and Distributed Computing* 137 (2020), 65–76. DOI: http://dx.doi.org/10.1016/j.jpdc.2019.10.004

[29] Mohammad Reza Samsami and Hossein Alimadad. 2020. Distributed deep reinforcement learning: An overview. arXiv:2011.11012. Retrieved from https://arxiv.org/abs/2011.11012

[30] Qiyue Yin, Tongtong Yu, Shengqi Shen, Jun Yang, Meijing Zhao, Wancheng Ni, Kaiqi Huang, Bin Liang, and Liang Wang. 2024. Distributed deep reinforcement learning: A survey and a multi-player multi-agent learning toolbox. *Machine Intelligence Research* 21, 3 (2024), 411–430.

[31] Ignasi Clavera, Jonas Rothfuss, John Schulman, Yasuhiro Fujita, Tamim Asfour, and Pieter Abbeel. 2018. Model-based reinforcement learning via meta-policy optimization. In *Proceedings of the Conference on Robot Learning*. PMLR, 617–629.

[32] Thanard Kurutach, Ignasi Clavera, Yan Duan, Aviv Tamar, and Pieter Abbeel. 2018. Model-ensemble trust-region policy optimization. arXiv:1802.10592. Retrieved from https://arxiv.org/abs/1802.10592

[33] Richard S. Sutton. 2018. Reinforcement learning: An introduction. *A Bradford Book* (2018).
[34] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. arXiv:1707.06347. Retrieved from https://arxiv.org/abs/1707.06347
[35] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the International Conference on Machine Learning*. PMLR, 1861–1870.
[36] Scott Fujimoto, Herke Hoof, and David Meger. 2018. Addressing function approximation error in actor-critic methods. In *Proceedings of the International Conference on Machine Learning*. PMLR, 1587–1596.
[37] Justin Fu, Aviral Kumar, Matthew Soh, and Sergey Levine. 2019. Diagnosing bottlenecks in deep q-learning algorithms. In *Proceedings of the International Conference on Machine Learning*. PMLR, 2021–2030.
[38] Aviral Kumar, Justin Fu, Matthew Soh, George Tucker, and Sergey Levine. 2019. Stabilizing off-policy q-learning via bootstrapping error reduction. In *Proceedings of the NeurIPS*.
[39] Cheng-Tao Chu, Sang Kim, Yi-An Lin, YuanYuan Yu, Gary Bradski, Kunle Olukotun, and Andrew Ng. 2006. Map-reduce for machine learning on multicore. In *Proceedings of the NeurIPS*.
[40] Martin Zinkevich, Markus Weimer, Lihong Li, and Alex Smola. 2010. Parallelized stochastic gradient descent. In *Proceedings of the NeurIPS*.
[41] Mu Li, David G. Andersen, Alexander J. Smola, and Kai Yu. 2014. Communication efficient distributed machine learning with the parameter server. In *Proceedings of the NeurIPS*. Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Q. Weinberger (Eds.), Vol. 27, Curran Associates, Inc.
[42] Jiawei Jiang, Bin Cui, Ce Zhang, and Lele Yu. 2017. Heterogeneity-aware distributed parameter servers. In *Proceedings of the ACM SIGMOD*. Association for Computing Machinery, New York, NY, USA, 463–478. DOI: http://dx.doi.org/10.1145/3035918.3035933
[43] Hao Zhang, Yuan Li, Zhijie Deng, Xiaodan Liang, Lawrence Carin, and Eric Xing. 2020. Autosync: Learning to synchronize for data-parallel distributed deep learning. In *Proceedings of the NeurIPS*. 906–917.
[44] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Marc'aurelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, et al. 2012. Large scale distributed deep networks. In *Proceedings of the NeurIPS*.
[45] Seunghak Lee, Jin Kyu Kim, Xun Zheng, Qirong Ho, Garth A. Gibson, and Eric P. Xing. 2014. On model parallelization and scheduling strategies for distributed machine learning. In *Proceedings of the NeurIPS*.
[46] Arpan Jain, Ammar Ahmad Awan, Asmaa M. Aljuhani, Jahanzeb Maqbool Hashmi, Quentin G. Anthony, Hari Subramoni, Dhableswar K. Panda, Raghu Machiraju, and Anil Parwani. 2020. Gems: Gpu-enabled memory-aware model-parallelism system for distributed dnn training. In *Proceedings of the SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.
[47] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. 2021. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*. 1–15.
[48] Zhenheng Tang, Shaohuai Shi, Xiaowen Chu, Wei Wang, and Bo Li. 2020. Communication-efficient distributed deep learning: A comprehensive survey. arXiv:2003.06307. Retrieved from https://arxiv.org/abs/2003.06307
[49] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, Hyouk Joong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. 2019. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *Proceedings of the NeurIPS*.
[50] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. 2019. Pipedream: Generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles* (*SOSP '19*). 1–15. DOI: http://dx.doi.org/10.1145/3341301.3359646
[51] Deepak Narayanan, Amar Phanishayee, Kaiyu Shi, Xie Chen, and Matei Zaharia. 2021. Memory-efficient pipeline-parallel dnn training. In *Proceedings of the International Conference on Machine Learning*. PMLR, 7937–7947.
[52] Ziyue Luo, Xiaodong Yi, Guoping Long, Shiqing Fan, Chuan Wu, Jun Yang, and Wei Lin. 2022. Efficient pipeline planning for expedited distributed DNN training. arXiv:2204.10562. Retrieved from https://arxiv.org/abs/2204.10562
[53] Alex Krizhevsky. 2014. One weird trick for parallelizing convolutional neural networks. arXiv:1404.5997. Retrieved from https://arxiv.org/abs/1404.5997
[54] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Łukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. 2016. Google's neural machine translation system: Bridging the gap between human and machine translation. arXiv:1609.08144. Retrieved from https://arxiv.org/abs/1609.08144

[55] Noam Shazeer, Youlong Cheng, Niki Parmar, Dustin Tran, Ashish Vaswani, Penporn Koanantakool, Peter Hawkins, HyoukJoong Lee, Mingsheng Hong, Cliff Young, Ryan Sepassi, and Blake Hechtman. 2018. Mesh-TensorFlow: Deep learning for supercomputers. In *Proceedings of the NeurIPS*. S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett (Eds.), Vol. 31, Curran Associates, Inc.

[56] Zhihao Jia, Matei Zaharia, and Alex Aiken. 2019. Beyond data and model parallelism for deep neural networks. arXiv:1807.05358. Retrieved from https://arxiv.org/abs/1807.05358

[57] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC*. 3505–3506. DOI: http://dx.doi.org/10.1109/SC41405.2020.00024

[58] Jay H. Park, Gyeongchan Yun, M. Yi Chang, Nguyen T. Nguyen, Seungmin Lee, Jaesik Choi, Sam H. Noh, and Young-ri Choi. 2020. {HetPipe}: Enabling large {DNN} training on (Whimpy) heterogeneous {GPU} clusters through integration of pipelined model parallelism and data parallelism. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC'20)*. 307–321.

[59] Zhengda Bian, Hongxin Liu, Boxiang Wang, Haichen Huang, Yongbin Li, Chuanrui Wang, Fan Cui, and Yang You. 2021. Colossal-AI: A unified deep learning system for large-scale parallel training. arXiv:2110.14883. Retrieved from https://arxiv.org/abs/2110.14883

[60] Aleksei Petrenko, Zhehui Huang, Tushar Kumar, Gaurav Sukhatme, and Vladlen Koltun. 2020. Sample factory: Egocentric 3D control from pixels at 100000 FPS with asynchronous reinforcement learning. In *Proceedings of the International Conference on Machine Learning*. PMLR, 7608–7618.

[61] Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Boron Yotam, Firoiu Vlad, Harley Tim, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. 2018. IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In *Proceedings of the International Conference on Machine Learning*. PMLR, 2263–2284.

[62] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, and Gavriel State. 2021. Isaac gym: High performance GPU-based physics simulation for robot learning. arXiv:2108.10470. Retrieved from https://arxiv.org/abs/2108.10470

[63] Bowen Baker, Ingmar Kanitscheider, Todor Markov, Yi Wu, Glenn Powell, Bob McGrew, and Igor Mordatch. 2020. Emergent tool use from multi-agent autocurricula. In *Proceedings of the International Conference on Learning Representations*.

[64] Raju Machupalli, Masum Hossain, and Mrinal Mandal. 2022. Review of ASIC accelerators for deep neural network. *Microprocessors and Microsystems* 89 (2022), 104441.

[65] Tianyi Chen, Kaiqing Zhang, Georgios B. Giannakis, and Tamer Basar. 2021. Communication-efficient policy gradient methods for distributed reinforcement learning. *IEEE Transactions on Control of Network Systems* 9, 2 (2021), 917–929. DOI: http://dx.doi.org/10.1109/TCNS.2021.3078100

[66] Felipe Petroski Such, Vashisht Madhavan, Edoardo Conti, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. 2017. Deep neuroevolution: Genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning. arXiv:1712.06567. Retrieved from https://arxiv.org/abs/1712.06567

[67] Dan Horgan, John Quan, David Budden, Gabriel Barth-Maron, Matteo Hessel, Hado Van Hasselt, and David Silver. 2018. Distributed prioritized experience replay. In *Proceedings of the International Conference on Learning Representations*. 1–19.

[68] Steven Kapturowski, Georg Ostrovski, John Quan, Remi Munos, and Will Dabney. 2018. Recurrent experience replay in distributed reinforcement learning. In *Proceedings of the International Conference on Learning Representations*.

[69] Arun Nair, Praveen Srinivasan, Sam Blackwell, Cagdas Alcicek, Rory Fearon, Alessandro De Maria, Vedavyas Panneershelvam, Mustafa Suleyman, Charles Beattie, Stig Petersen, Shane Legg, Volodymyr Mnih, Koray Kavukcuoglu, and David Silver. 2015. Massively parallel methods for deep reinforcement learning. arXiv:1507.04296. Retrieved from https://arxiv.org/abs/1507.04296

[70] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. 2016. Asynchronous methods for deep reinforcement learning. In *Proceedings of the International Conference on Machine Learning*. PMLR, 1928–1937.

[71] Adam Stooke and Pieter Abbeel. rlpyt : A research code base for deep reinforcement learning in PyTorch. arXiv:1909.01500v2. Retrieved from https://arxiv.org/abs/1909.01500v2

[72] Erik Wijmans, Abhishek Kadian, Ari Morcos, Stefan Lee, Irfan Essa, Devi Parikh, Manolis Savva, and Dhruv Batra. 2020. DD-PPO: Learning near-perfect PointGoal navigators from 2.5 billion frames. In *Proceedings of the International Conference on Learning Representations*. 1–21.

[73] Eric Liang, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph Gonzalez, Michael Jordan, and Ion Stoica. 2018. RLlib: Abstractions for distributed reinforcement learning. In *Proceedings of the International Conference on Machine Learning*. PMLR, 3053–3062.

[74] Youjie Li, Iou Jen Liu, Yifan Yuan, Deming Chen, Alexander Schwing, and Jian Huang. 2019. Accelerating distributed reinforcement learning with in-switch computing. *Proceedings - International Symposium on Computer Architecture* (2019), 279–291. DOI: http://dx.doi.org/10.1145/3307650.3322259

[75] Adam Stooke and Pieter Abbeel. 2018. Accelerated methods for deep reinforcement learning. arXiv:1803.02811. Retrieved from https://arxiv.org/abs/1803.02811

[76] Amir Yazdanbakhsh, Junchao Chen, and Yu Zheng. 2020. Menger: Massively large-scale distributed reinforcement learning. *NeurIPS, Beyond Backpropagation Workshop* 3 (2020). https://research.google/pubs/pub49803/

[77] Lasse Espeholt, Raphaël Marinier, Piotr Stanczyk, Ke Wang, and Marcin Michalski. 2020. Seed rl: Scalable and efficient deep-rl with accelerated central inference. In *Proceedings of the International Conference on Learning Representations*.

[78] Zhiyu Mei, Wei Fu, Guangju Wang, Huanchen Zhang, and Yi Wu. 2024. SRL: Scaling distributed reinforcement learning to over ten thousand cores. In *Proceedings of the International Conference on Learning Representations*.

[79] Zechu Li, Tao Chen, Zhang-Wei Hong, Anurag Ajay, and Pulkit Agrawal. 2023. Parallel Q-Learning: Scaling Off-policy reinforcement learning under massively parallel simulation. In *Proceedings of the International Conference on Machine Learning*. PMLR, 19440–19459.

[80] Yunzhi Zhang, Ignasi Clavera, Boren Tsai, and Pieter Abbeel. 2020. Asynchronous methods for model-based reinforcement learning. In *Proceedings of the Conference on Robot Learning*. PMLR, 1338–1347.

[81] Mahmoud Assran, Joshua Romoff, Nicolas Ballas, Joelle Pineau, and Michael Rabbat. 2019. Gossip-based actor-learner architectures for deep reinforcement learning. In *Proceedings of the NeurIPS*.

[82] Adwaitvedant Mathkar and Vivek S. Borkar. 2017. Distributed reinforcement learning via gossip. *IEEE Transactions on Automatic Control* 62, 3 (2017), 1465–1470. DOI: http://dx.doi.org/10.1109/TAC.2016.2585302

[83] Xingyu Sha, Jiaqi Zhang, Keyou You, Kaiqing Zhang, and Tamer Başar. 2022. Fully asynchronous policy evaluation in distributed reinforcement learning over networks. *Automatica* 136 (2022), 110092.

[84] Matteo Hessel, Joseph Modayil, Hado Van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. 2018. Rainbow: Combining improvements in deep reinforcement learning. arXiv:1710.02298. Retrieved from https://arxiv.org/abs/1710.02298

[85] Audrūnas Gruslys, Will Dabney, Mohammad Gheshlaghi Azar, Bilal Piot, Marc G. Bellemare, and Rémi Munos. 2018. The reactor: A fast and sample-efficient actor-critic agent for reinforcement learning. In *Proceedings of the International Conference on Learning Representations*. 1–18.

[86] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. 2015. Trust region policy optimization. In *Proceedings of the International Conference on Machine Learning*. PMLR, 1889–1897.

[87] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2015. Continuous control with deep reinforcement learning. arXiv:1509.02971. Retrieved from https://arxiv.org/abs/1509.02971

[88] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. arXiv:1707.06347. Retrieved from https://arxiv.org/abs/1707.06347

[89] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine. 2018. Soft actor-critic algorithms and applications. arXiv:1812.05905. Retrieved from https://arxiv.org/abs/1812.05905

[90] Yuandong Tian, Qucheng Gong, Wenling Shang, Yuxin Wu, and C. Lawrence Zitnick. 2017. ELF: An extensive, lightweight and flexible research platform for real-time strategy games. In *Proceedings of the NeurIPS*. 2660–2670.

[91] C. Daniel Freeman, Erik Frey, Anton Raichuk, Sertan Girgin, Igor Mordatch, and Olivier Bachem. 2021. Brax–a differentiable physics engine for large scale rigid body simulation. arXiv:2106.13281. Retrieved from https://arxiv.org/abs/2106.13281

[92] Radhika Mittal, Vinh The Lam, Nandita Dukkipati, Emily Blem, Hassan Wassel, Monia Ghobadi, Amin Vahdat, Yaogong Wang, David Wetherall, and David Zats. 2015. TIMELY: RTT-based congestion control for the datacenter. *ACM SIGCOMM Computer Communication Review* 45, 4 (2015), 537–550.

[93] Zhihong Liu, Qi Zhang, Reaz Ahmed, Raouf Boutaba, Yaping Liu, and Zhenghu Gong. 2016. Dynamic resource allocation for MapReduce with partitioning skew. *IEEE Transactions on Computers* 65, 11 (2016), 3304–3317.

[94] Steven Dalton and Iuri Frosio. 2020. Accelerating reinforcement learning through GPU Atari emulation. In *Proceedings of the NeurIPS*.

[95] Jing Guo, Zihao Chang, Sa Wang, Haiyang Ding, Yihui Feng, Liang Mao, and Yungang Bao. 2019. Who limits the resource efficiency of my datacenter: An analysis of alibaba datacenter traces. In *Proceedings of the International Symposium on Quality of Service*. 1–10.

[96] Jacky Liang, Viktor Makoviychuk, Ankur Handa, Nuttapong Chentanez, Miles Macklin, and Dieter Fox. 2018. GPU-accelerated robotic simulation for distributed reinforcement learning. arXiv:1810.05762. Retrieved from https://arxiv.org/abs/1810.05762

[97] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. 2017. Evolution strategies as a scalable alternative to reinforcement learning. arXiv:1703.03864. Retrieved from https://arxiv.org/abs/1703.03864

[98] Brennan Shacklett, Erik Wijmans, Aleksei Petrenko, Manolis Savva, Dhruv Batra, Vladlen Koltun, and Kayvon Fatahalian. 2021. Large batch simulation for deep reinforcement learning. In *Proceedings of the International Conference on Learning Representations*.

[99] Xue Bin Peng, Ze Ma, Pieter Abbeel, Sergey Levine, and Angjoo Kanazawa. 2021. Amp: Adversarial motion priors for stylized physics-based character control. *ACM Transactions on Graphics* 40, 4 (2021), 1–20.

[100] C. Daniel Freeman, Erik Frey, Anton Raichuk, Sertan Girgin, Igor Mordatch, and Olivier Bachem. 2021. Brax – a differentiable physics engine for large scale rigid body simulation. arXiv:2106.13281. Retrieved from https://arxiv.org/abs/2106.13281

[101] Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. 2022. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Proceedings of the 5th Conference on Robot Learning*. Aleksandra Faust, David Hsu, and Gerhard Neumann (Eds.), Proceedings of Machine Learning Research, Vol. 164, PMLR, 91–100.

[102] Antonio Loquercio, Elia Kaufmann, René Ranftl, Matthias Müller, Vladlen Koltun, and Davide Scaramuzza. 2021. Learning high-speed flight in the wild. *Science Robotics* 6, 59 (2021), eabg5810.

[103] Jeffrey Dean and Sanjay Ghemawat. 2008. MapReduce: Simplified data processing on large clusters. *Communications of the ACM* 51, 1 (2008), 107–113.

[104] Yuxi Li and Dale Schuurmans. 2011. Mapreduce for parallel reinforcement learning. In *Proceedings of the European Workshop on Reinforcement Learning*. Springer, 309–320.

[105] Eric Liang, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Joseph Gonzalez, Ken Goldberg, and Ion Stoica. 2017. Ray rllib: A composable and scalable reinforcement learning library. arXiv:1712.09381. Retrieved from https://arxiv.org/abs/1712.09381

[106] Horia Mania, Aurelia Guy, and Benjamin Recht. 2018. Simple random search provides a competitive approach to reinforcement learning. arXiv:1803.07055. Retrieved from https://arxiv.org/abs/1803.07055

[107] Igor Adamski, Robert Adamski, Tomasz Grel, Adam Jędrych, Kamil Kaczmarek, and Henryk Michalewski. 2018. Distributed deep reinforcement learning: Learn how to play atari games in 21 minutes. In *Proceedings of the ISC High Performance 2018, Frankfurt, Germany*. Springer, 370–388.

[108] Nicolas Heess, Dhruva T. B., Srinivasan Sriram, Jay Lemmon, Josh Merel, Greg Wayne, Yuval Tassa, Tom Erez, Ziyu Wang, S. M. Ali Eslami, Martin Riedmiller, and David Silver. 2017. Emergence of locomotion behaviours in rich environments. arXiv:1707.02286. Retrieved from https://arxiv.org/abs/1707.02286

[109] P. Olicy G. Radients, Gabriel Barth-maron, Matthew W. Hoffman, David Budden, Will Dabney, Dan Horgan, Dhruva Tb, Alistair Muldal, Nicolas Heess, and Timothy Lillicrap. Distributed distributional deterministic policy gradients. In *Proceedings of the International Conference on Learning Representations*. 1–16.

[110] Mohammad Babaeizadeh, Iuri Frosio, Stephen Tyree, Jason Clemons, and Jan Kautz. 2017. Reinforcement learning through asynchronous advantage actor-critic on a GPU. In *Proceedings of the International Conference on Learning Representations*. 1–12.

[111] Alfredo V. Clemente, Humberto N. Castejón, and Arjun Chandra. 2017. Efficient parallel methods for deep reinforcement learning. arXiv:1705.04862. Retrieved from https://arxiv.org/abs/1705.04862

[112] Huanzhou Zhu, Bo Zhao, Gang Chen, Weifeng Chen, Yijie Chen, Liang Shi, Yaodong Yang, Peter Pietzuch, and Lei Chen. 2023. {MSRL}: Distributed reinforcement learning with dataflow fragments. In *Proceedings of the 2023 USENIX Annual Technical Conference (USENIX ATC'23)*. 977–993.

[113] Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Dębiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, et al. 2019. Dota 2 with large scale deep reinforcement learning. arXiv:1912.06680. Retrieved from https://arxiv.org/abs/1912.06680

[114] Yixuan Mei, Jiaxuan Gao, Weirui Ye, Shaohuai Liu, Yang Gao, and Yi Wu. 2023. Speedyzero: Mastering atari with limited data and time. In *Proceedings of the International Conference on Learning Representations*.

[115] Jiang Su, Jianxiong Liu, David B. Thomas, and Peter Y. K. Cheung. 2017. Neural network based reinforcement learning acceleration on FPGA platforms. *ACM SIGARCH Computer Architecture News* 44, 4 (2017), 68–73. DOI: http://dx.doi.org/10.1145/3039902.3039915

[116] Shengjia Shao and Wayne Luk. 2017. Customised pearlmutter propagation: A hardware architecture for trust region policy optimisation. In *Proceedings of the 2017 27th International Conference on Field Programmable Logic and Applications, FPL 2017*. DOI: http://dx.doi.org/10.23919/FPL.2017.8056789

[117] Ce Guo, Wayne Luk, Stanley Qing Shui Loh, Alexander Warren, and Joshua Levine. 2019. Customisable control policy learning for robotics. In *Proceedings of the International Conference on Application-Specific Systems, Architectures and Processors*. 91–98. DOI: http://dx.doi.org/10.1109/ASAP.2019.00-24

[118] Hyungmin Cho, Pyeongseok Oh, Jiyoung Park, Wookeun Jung, and Jaejin Lee. 2019. FA3C: FPGA-accelerated deep reinforcement learning. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems - ASPLOS*. 499–513. DOI: http://dx.doi.org/10.1145/3297858.3304058

[119] Yuan Meng, Sanmukh Kuppannagari, and Viktor Prasanna. 2020. Accelerating proximal policy optimization on CPU-FPGA heterogeneous platforms. In *Proceedings of the 28th IEEE International Symposium on Field-Programmable Custom Computing Machines, FCCM 2020*. 19–27. DOI: http://dx.doi.org/10.1109/FCCM48280.2020.00012

[120] Yuan Meng, Chi Zhang, and Viktor Prasanna. 2022. FPGA acceleration of deep reinforcement learning using on-chip replay management. In *Proceedings of the 19th ACM International Conference on Computing Frontiers*. (2022), 40–48. DOI: http://dx.doi.org/10.1145/3528416.3530227

[121] Scott Reed, Konrad Zolna, Emilio Parisotto, Sergio Gomez Colmenarejo, Alexander Novikov, Gabriel Barth-Maron, Mai Gimenez, Yury Sulsky, Jackie Kay, Jost Tobias Springenberg, Tom Eccles, Jake Bruce, Ali Razavi, Ashley Edwards, Nicolas Heess, Yutian Chen, Raia Hadsell, Oriol Vinyals, Mahyar Bordbar, and Nando de Freitas. 2022. A generalist agent. arXiv:2205.06175. Retrieved from https://arxiv.org/abs/2205.06175

[122] Jiale Zhi, Rui Wang, Jeff Clune, and Kenneth O. Stanley. 2020. Fiber: A platform for efficient development and distributed training for reinforcement learning and population-based methods. arXiv:2003.11164. Retrieved from https://arxiv.org/abs/2003.11164

[123] Lucileide M. D. Da Silva, Matheus F. Torquato, and Marcelo A. C. Fernandes. 2018. Parallel implementation of reinforcement learning q-learning technique for fpga. *IEEE Access* 7 (2018), 2782–2798.

[124] Rachit Rajat, Yuan Meng, Sanmukh Kuppannagari, Ajitesh Srivastava, Viktor Prasanna, and Rajgopal Kannan. 2020. Qtaccel: A generic fpga based design for q-table based reinforcement learning accelerators. In *Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 323–323.

[125] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. 2018. Ray: A distributed framework for emerging AI applications. In *OSDI'18: Proceedings of the 13th USENIX conference on Operating Systems Design and Implementation*.

[126] Hanfei Yu, Jian Li, Yang Hua, Xu Yuan, and Hao Wang. 2024. Cheaper and faster: Distributed deep reinforcement learning with serverless computing. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 16539–16547.

[127] Yuan Meng, Michael Kinsner, Deshanand Singh, Mahesh Iyer, and Viktor Prasanna. 2024. PEARL: Enabling portable, productive, and high-performance deep reinforcement learning using heterogeneous platforms. In *Proceedings of the 21st ACM International Conference on Computing Frontiers*. 41–50.

[128] Albert Bou and Gianni De Fabritiis. 2020. PyTorchRL: Modular and distributed reinforcement learning in PyTorch. arXiv:2007.02622. Retrieved from https://arxiv.org/abs/2007.02622

[129] Xinghao Pan, Jianmin Chen, Rajat Monga, Samy Bengio, and Rafal Jozefowicz. 2017. Revisiting distributed synchronous SGD. arXiv:1702.05800. Retrieved from https://arxiv.org/abs/1702.05800

[130] Xiangru Lian, Wei Zhang, Ce Zhang, and Ji Liu. 2018. Asynchronous decentralized parallel stochastic gradient descent. In *Proceedings of the International Conference on Machine Learning*. PMLR, 4745–4767.

[131] Qirong Ho, James Cipar, Henggang Cui, Jin Kyu Kim, Seunghak Lee, Phillip B. Gibbons, Garth A. Gibson, Gregory R. Ganger, and Eric P. Xing. 2013. More effective distributed ML via a stale synchronous parallel parameter server. In *Proceedings of the NeurIPS*. 1–9.

[132] Youjie Li, Mingchao Yu, Songze Li, Salman Avestimehr, Nam Sung Kim, and Alexander Schwing. 2018. PIPE-SGD: A decentralized pipelined SGD framework for distributed deep net training. In *Proceedings of the NeurIPS*. 8045–8056.

[133] Alfredo V. Clemente, Humberto N. Castejón, and Arjun Chandra. 2017. Efficient parallel methods for deep reinforcement learning. arXiv:1705.04862. Retrieved from https://arxiv.org/abs/1705.04862

[134] Charles Reiss, Alexey Tumanov, Gregory R. Ganger, Randy H. Katz, and Michael A. Kozuch. 2012. Heterogeneity and dynamicity of clouds at scale: Google trace analysis. In *Proceedings of the 3rd ACM Symposium on Cloud Computing*. 1–13.

[135] Iou-Jen Liu, Raymond Yeh, and Alexander Schwing. 2020. High-throughput synchronous deep rl. *Advances in Neural Information Processing Systems* 33 (2020), 17070–17080.

[136] Xing Zhao, Aijun An, Junfeng Liu, and Bao Xin Chen. 2019. Dynamic stale synchronous parallel distributed training for deep learning. In *Proceedings of the 2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS'19)*. IEEE, 1507–1517.

[137] Haifeng Sun, Zhiyi Gui, Song Guo, Qi Qi, Jingyu Wang, and Jianxi Liao. 2021. GSSP: Eliminating stragglers through grouping synchronous for distributed deep learning in heterogeneous cluster. *IEEE Transactions on Cloud Computing* 14, 8 (2021), 2637–2648. DOI: http://dx.doi.org/10.1109/TCC.2021.3062398

[138] Simon Schmitt, Matteo Hessel, and Karen Simonyan. 2020. Off-policy actor-critic with shared experience replay. In *Proceedings of the International Conference on Machine Learning*. PMLR, 8545–8554.

[139] Alexandre Borges and Arlindo Oliveira. 2021. Combining off and on-policy training in model-based reinforcement learning. arXiv:2102.12194. Retrieved from https://arxiv.org/abs/2102.12194

[140] Yunhao Tang. 2021. Guiding evolutionary strategies with off-policy actor-critic. In *Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems*. 1317–1325.
[141] Edoardo Conti, Joel Lehman, and Kenneth O. Stanley. 2018. Improving exploration in evolution strategies for deep reinforcement learning via a population of novelty-seeking agents. In *Proceedings of the 32nd Conference on Neural Information Processing Systems* 31 (2018).
[142] Kenneth O. Stanley, Jeff Clune, Joel Lehman, and Risto Miikkulainen. 2019. Designing neural networks through neuroevolution. *Nature Machine Intelligence* 1, 1 (2019), 24–35.
[143] Shauharda Khadka. 2019. Evolution-guided policy gradient in reinforcement learning. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems*. 1188–1200.
[144] Daan Wierstra, Tom Schaul, Tobias Glasmachers, Yi Sun, Jan Peters, and Jürgen Schmidhuber. 2014. Natural evolution strategies. *The Journal of Machine Learning Research* 15, 1 (2014), 949–980.
[145] Verena Heidrich-Meisner and Christian Igel. 2009. Uncertainty handling CMA-ES for reinforcement learning. In *Proceedings of the 11th Annual Conference on Genetic and Evolutionary Computation*. 1211–1218.
[146] Manoj Kumar, Dr Husain, Naveen Upreti, Deepti Gupta, et al. 2010. Genetic algorithm: Review and application. *International Journal of Information Technology and Knowledge Management* 2, 2 (2010), 451–454.
[147] Kenneth O. Stanley and Risto Miikkulainen. 2002. Evolving neural networks through augmenting topologies. *Evolutionary Computation* 10, 2 (2002), 99–127.
[148] Kenneth O. Stanley and Risto Miikkulainen. 2002. Efficient reinforcement learning through evolving neural network topologies. In *Proceedings of the 4th Annual Conference on Genetic and Evolutionary Computation*. 569–577.
[149] Kenneth O. Stanley, David B. D'Ambrosio, and Jason Gauci. 2009. A hypercube-based encoding for evolving large-scale neural networks. *Artificial Life* 15, 2 (2009), 185–212.
[150] Barret Zoph and Quoc V. Le. 2017. Neural architecture search with reinforcement learning. In *Proceedings of the International Conference on Learning Representations*.
[151] Esteban Real, Sherry Moore, Andrew Selle, Saurabh Saxena, Yutaka Leon Suematsu, Jie Tan, Quoc V. Le, and Alexey Kurakin. 2017. Large-scale evolution of image classifiers. In *Proceedings of the International Conference on Machine Learning*. PMLR, 2902–2911.
[152] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V. Le. 2019. Regularized evolution for image classifier architecture search. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 4780–4789.
[153] Gustavo-Adolfo Vargas-Hakim, Efren Mezura-Montes, and Hector-Gabriel Acosta-Mesa. 2021. A review on convolutional neural network encodings for neuroevolution. *IEEE Transactions on Evolutionary Computation* 26, 1 (2021), 12–27.
[154] Alejandro Baldominos, Yago Saez, and Pedro Isasi. 2018. Evolutionary convolutional neural networks: An application to handwriting recognition. *Neurocomputing* 283 (2018), 38–52.
[155] Miao Zhang, Huiqi Li, Shirui Pan, Juan Lyu, Steve Ling, and Steven Su. 2021. Convolutional neural networks-based lung nodule classification: A surrogate-assisted evolutionary algorithm for hyperparameter optimization. *IEEE Transactions on Evolutionary Computation* 25, 5 (2021), 869–882.
[156] Tanmay Gangwani and Jian Peng. 2018. Policy optimization by genetic distillation. In *Proceedings of the International Conference on Learning Representations*.
[157] Prafulla Dhariwal, Christopher Hesse, Oleg Klimov, Alex Nichol, Matthias Plappert, Alec Radford, John Schulman, Szymon Sidor, Yuhuai Wu, and Peter Zhokhov. 2017. OpenAI Baselines. https://github.com/openai/baselines. (2017).
[158] Antoine Cully, Jeff Clune, Danesh Tarapore, and Jean-Baptiste Mouret. 2015. Robots that can adapt like animals. *Nature* 521, 7553 (2015), 503–507.
[159] Justin K. Pugh, Lisa B. Soros, and Kenneth O. Stanley. 2016. Quality diversity: A new frontier for evolutionary computation. *Frontiers in Robotics and AI* 3 (2016), 202845.
[160] Edoardo Conti, Vashisht Madhavan, Felipe Petroski Such, Joel Lehman, Kenneth Stanley, and Jeff Clune. 2018. Improving exploration in evolution strategies for deep reinforcement learning via a population of novelty-seeking agents. In *Proceedings of the NeurIPS*.
[161] Ethan C. Jackson and Mark Daley. 2019. Novelty search for deep reinforcement learning policy network weights by action sequence edit metric distance. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*. 173–174.
[162] Matthias Plappert, Rein Houthooft, Prafulla Dhariwal, Szymon Sidor, Richard Y. Chen, Xi Chen, Tamim Asfour, Pieter Abbeel, and Marcin Andrychowicz. 2018. Parameter space noise for exploration. In *Proceedings of the International Conference on Learning Representations*.
[163] Meire Fortunato, Mohammad Gheshlaghi Azar, Bilal Piot, Jacob Menick, Ian Osband, Alex Graves, Vlad Mnih, Remi Munos, Demis Hassabis, Olivier Pietquin, et al. 2018. Noisy networks for exploration. In *Proceedings of the International Conference on Learning Representations*.

[164] Francisco E. Fernandes Jr and Gary G. Yen. 2021. Pruning deep convolutional neural networks architectures with evolution strategy. *Information Sciences* 552 (2021), 29–47.
[165] Gustavo Adolfo Vargas-Hákim, Efrén Mezura-Montes, and Héctor Gabriel Acosta-Mesa. 2022. A review on convolutional neural network encodings for neuroevolution. *IEEE Transactions on Evolutionary Computation* 26, 1 (2022), 12–27. DOI: http://dx.doi.org/10.1109/TEVC.2021.3088631
[166] Filipe Assunção, Nuno Lourenço, Bernardete Ribeiro, and Penousal Machado. 2020. Incremental evolution and development of deep artificial neural networks. In *Proceedings of the European Conference on Genetic Programming (Part of EvoStar)*. Springer, 35–51.
[167] Francisco Erivaldo Fernandes Junior and Gary G. Yen. 2019. Particle swarm optimization of deep neural networks architectures for image classification. *Swarm and Evolutionary Computation* 49 (2019), 62–74.
[168] Ye-Qun Wang, Chun-Hua Chen, Jun Zhang, and Zhi-Hui Zhan. 2022. Dropout topology-assisted bidirectional learning particle swarm optimization for neural architecture search. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*. 93–96.
[169] Ashraf Darwish, Aboul Ella Hassanien, and Swagatam Das. 2020. A survey of swarm and evolutionary computing approaches for deep learning. *Artificial Intelligence Review* 53, 3 (2020), 1767–1812.
[170] Hanxiao Liu, Karen Simonyan, and Yiming Yang. 2018. DARTS: Differentiable architecture search. In *Proceedings of the International Conference on Learning Representations*.
[171] Liam Li and Ameet Talwalkar. 2020. Random search and reproducibility for neural architecture search. In *Proceedings of the Uncertainty in Artificial Intelligence*. PMLR, 367–377.
[172] Miao Zhang, Huiqi Li, Shirui Pan, Xiaojun Chang, and Steven Su. 2020. Overcoming multi-model forgetting in one-shot NAS with diversity maximization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 7809–7818.
[173] Kirthevasan Kandasamy, Willie Neiswanger, Jeff Schneider, Barnabas Poczos, and Eric P. Xing. 2018. Neural architecture search with bayesian optimisation and optimal transport. In *Proceedings of the NeurIPS*.
[174] Chenxi Liu, Barret Zoph, Maxim Neumann, Jonathon Shlens, Wei Hua, Li-Jia Li, Li Fei-Fei, Alan Yuille, Jonathan Huang, and Kevin Murphy. 2018. Progressive neural architecture search. In *Proceedings of the European Conference on Computer Vision (ECCV'18)*. 19–34.
[175] Karen Simonyan and Andrew Zisserman. 2015. Very deep convolutional networks for large-scale image recognition. In *Proceedings of the International Conference on Learning Representations*.
[176] Yongchun Zhu, Fuzhen Zhuang, Jindong Wang, Guolin Ke, Jingwu Chen, Jiang Bian, Hui Xiong, and Qing He. 2020. Deep subdomain adaptation network for image classification. *IEEE Transactions on Neural Networks and Learning Systems* 32, 4 (2020), 1713–1722.
[177] Kuan-Hung Shih, Ching-Te Chiu, Jiou-Ai Lin, and Yen-Yu Bu. 2019. Real-time object detection with reduced region proposal network via multi-feature concatenation. *IEEE Transactions on Neural Networks and Learning Systems* 31, 6 (2019), 2164–2173.
[178] Dingwen Zhang, Junwei Han, Long Zhao, and Tao Zhao. 2020. From discriminant to complete: Reinforcement searching-agent learning for weakly supervised object detection. *IEEE Transactions on Neural Networks and Learning Systems* 31, 12 (2020), 5549–5560.
[179] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. 2016. Google's neural machine translation system: Bridging the gap between human and machine translation. arXiv:1609.08144. Retrieved from https://arxiv.org/abs/1609.08144
[180] Biao Zhang, Deyi Xiong, Jun Xie, and Jinsong Su. 2020. Neural machine translation with GRU-gated attention model. *IEEE Transactions on Neural Networks and Learning Systems* 31, 11 (2020), 4688–4698.
[181] Jason Liang, Elliot Meyerson, and Risto Miikkulainen. 2018. Evolutionary architecture search for deep multitask networks. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 466–473.
[182] Eunwoo Kim, Chanho Ahn, and Songhwai Oh. 2021. Auto-VirtualNet: Cost-adaptive dynamic architecture search for multi-task learning. *Neurocomputing* 442 (2021), 116–124.
[183] Alexis Asseman, Nicolas Antoine, and Ahmet S. Ozcan. 2021. Accelerating deep neuroevolution on distributed FPGAs for reinforcement learning problems. *ACM Journal on Emerging Technologies in Computing Systems* 17, 2 (2021), 1–17. DOI: http://dx.doi.org/10.1145/3425500
[184] Sergio Guadarrama, Anoop Korattikara, Oscar Ramirez, Pablo Castro, Ethan Holly, Sam Fishman, Ke Wang, Ekaterina Gonina, Neal Wu, Efi Kokiopoulou, Luciano Sbaiz, Jamie Smith, Gábor Bartók, Jesse Berent, Chris Harris, Vincent Vanhoucke, and Eugene Brevdo. 2018. TF-Agents: A Library for Reinforcement Learning in TensorFlow. https://github.com/tensorflow/agents. (2018). https://github.com/tensorflow/agents [Online; accessed 25-June-2019].
[185] Jason Gauci, Edoardo Conti, Yitao Liang, Kittipat Virochsiri, Yuchen He, Zachary Kaden, Vivek Narayanan, Xiaohui Ye, Zhengxing Chen, and Scott Fujimoto. 2019. Horizon : Facebook ' s open source applied reinforcement learning platform. arXiv:1811.00260v5. Retrieved from https://arxiv.org/abs/1811.00260v5

[186] Baidu. 2022. A flexible, distributed and object-oriented programming reinforcement learning framework. *github website* (2022). https://github.com/PaddlePaddle/PARL

[187] Matt Hoffman, Bobak Shahriari, John Aslanides, Gabriel Barth-Maron, Feryal Behbahani, Tamara Norman, Abbas Abdolmaleki, Albin Cassirer, Fan Yang, Kate Baumli, Sarah Henderson, Alex Novikov, Sergio Gómez Colmenarejo, Serkan Cabi, Caglar Gulcehre, Tom Le Paine, Andrew Cowie, Ziyu Wang, Bilal Piot, and Nando de Freitas. 2020. Acme: A research framework for distributed reinforcement learning. arXiv:2006.00979. Retrieved from https://arxiv.org/abs/2006.00979

[188] Yasuhiro Fujita, Prabhat Nagarajan, Toshiki Kataoka, and Takahiro Ishikawa. 2021. ChainerRL: A deep reinforcement learning library. *Journal of Machine Learning Research* 22, 77 (2021), 1–14. Retrieved from http://jmlr.org/papers/v22/20-376.html

[189] Jiayi Weng, Huayu Chen, Dong Yan, Kaichao You, Alexis Duburcq, Minghao Zhang, Yi Su, Hang Su, and Jun Zhu. 2022. Tianshou: A highly modularized deep reinforcement learning library. *Journal of Machine Learning Research* 23, 267 (2022), 1–6. Retrieved from http://jmlr.org/papers/v23/21-1127.html

[190] Albert Bou, Matteo Bettini, Sebastian Dittert, Vikash Kumar, Shagun Sodhani, Xiaomeng Yang, Gianni De Fabritiis, and Vincent Moens. 2023. TorchRL: A Data-driven Decision-making Library for PyTorch. arXiv:2306.00577. Retrieved from https://arxiv.org/abs/2306.00577

[191] Abu Sebastian, Manuel Le Gallo, Riduan Khaddam-Aljameh, and Evangelos Eleftheriou. 2020. Memory devices and applications for in-memory computing. *Nature Nanotechnology* 15, 7 (2020), 529–544.

[192] Tianqi Wang, Tong Geng, Ang Li, Xi Jin, and Martin Herbordt. 2020. FPDeep: Scalable acceleration of CNN training on deeply-pipelined FPGA clusters. *IEEE Transactions on Computers* 69, 8 (2020), 1143–1158.

[193] Arjun Rao, Philipp Plank, Andreas Wild, and Wolfgang Maass. 2022. A long short-term memory for AI applications in spike-based neuromorphic hardware. *Nature Machine Intelligence* 4, 5 (2022), 467–479.

[194] Zijian Hu, Xiaoguang Gao, Kaifang Wan, Qianglong Wang, and Yiwei Zhai. 2023. Asynchronous curriculum experience replay: A deep reinforcement learning approach for UAV autonomous motion control in unknown dynamic environments. *IEEE Transactions on Vehicular Technology* 72, 11 (2023), 13985–14001.

[195] Xu-Hui Liu, Zhenghai Xue, Jingcheng Pang, Shengyi Jiang, Feng Xu, and Yang Yu. 2021. Regret minimization experience replay in off-policy reinforcement learning. *Advances in Neural Information Processing Systems* 34 (2021), 17604–17615.

[196] Clemens Schwarke, Victor Klemm, Matthijs van der Boon, Marko Bjelonic, and Marco Hutter. 2023. Curiosity-driven learning for joint locomotion and manipulation tasks. In *Proceedings of the 7th Annual Conference on Robot Learning*.

[197] Shuang Wu, Jian Yao, Haobo Fu, Ye Tian, Chao Qian, Yaodong Yang, Qiang Fu, and Yang Wei. 2022. Quality-similar diversity via population based reinforcement learning. In *Proceedings of the International Conference on Learning Representations*.

[198] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. 2022. Training a helpful and harmless assistant with reinforcement learning from human feedback. arXiv:2204.05862. Retrieved from https://arxiv.org/abs/2204.05862

[199] Kun Chu, Xufeng Zhao, Cornelius Weber, Mengdi Li, and Stefan Wermter. 2023. Accelerating reinforcement learning of robotic manipulations via feedback from large language models. arXiv:2311.02379. Retrieved from https://arxiv.org/abs/2311.02379

[200] Yuji Cao, Huan Zhao, Yuheng Cheng, Ting Shu, Guolong Liu, Gaoqi Liang, Junhua Zhao, and Yun Li. 2024. Survey on large language model-enhanced reinforcement learning: Concept, taxonomy, and methods. arXiv:2404.00282. Retrieved from https://arxiv.org/abs/2404.00282

Received 11 November 2023; revised 26 August 2024; accepted 28 October 2024